

Computer Architecture and Operating Systems — Block 1 to 3 MALAYALAM

Block 1: Basic Functional Architecture

Unit 1: Functional Units and Bus

1.1.1 കമ്പ്യൂട്ടറുകളുടെ തലമുറകൾ (Generations of Computers)

Theory

കമ്പ്യൂട്ടറുകളുടെ **generation** എന്ന പദം കാലക്രമത്തിൽ കമ്പ്യൂട്ടറിന്റെ **hardware-ഉം software-ഉം എങ്ങനെ വികസിച്ചു** എന്നതിനെ സൂചിപ്പിക്കുന്നു. ഓരോ തലമുറയും **switching technology, size, speed, cost, programming style** എന്നിവയിലെ മാറ്റങ്ങളാൽ തിരിച്ചറിയപ്പെടുന്നു.

John von Neumann-ന്റെ അടിസ്ഥാന **organisational model** ആയ **input, output, memory, ALU, control** എന്നിവ, ഉപകരണങ്ങളുടെ **രൂപം, വലിപ്പം, performance** എന്നിവ വ്യത്യസ്തമായിരുന്നാലും, എല്ലാ തലമുറകളിലും പൊതുവായി നിലനിൽക്കുന്നു.

Important Points

Generation	Period	Technology	Features	Examples
First	1940–1956	Vacuum tubes; magnetic drums	വളരെ വലുത്, ചെലവേറിയത്, കൂടുതൽ power/heat; mechanical devices- നേക്കാൾ വേഗതയേറിയത്	UNIVAC, ENIAC, EDVAC
Second	1956–1963	Transistors	ചെറുത്, വേഗതയേറിയത്, കുറവ് heat;	IBM 1400 series, IBM 7090/7094,

Generation	Period	Technology	Features	Examples
			COBOL, FORTRAN	UNIVAC 1107, CDC 3600
Third	1964–1971	Integrated Circuits (ICs)	കൂടുതൽ reliable, വേഗതയേറിയത്, ചെറുത്, വിലകുറഞ്ഞത്, കുറവ് maintenance	IBM 370, PDP-11, IBM System/360, UNIVAC 1108, Honeywell-6000
Fourth	1971–2010	VLSI / microprocessors	PCs, laptops, handhelds; ഒരു silicon chip-ൽ നിരവധി ICs	Apple, CRAY-1
Fifth	Present & future	Artificial Intelligence (AI)	മനുഷ്യന്റെ thinking/acting അനുകരിക്കുന്ന systems	PARAM 10000, IBM notebooks

For Exam

Computer generations hardware-ഉം software-ഉം എങ്ങനെ evolve ചെയ്തു എന്ന് classify ചെയ്യുന്നു. First generation vacuum tubes ഉപയോഗിച്ചു (ENIAC, UNIVAC). Second generation transistors-ഉം COBOL, FORTRAN പോലുള്ള high-level languages-ഉം ഉപയോഗിച്ചു. Third generation integrated circuits ഉപയോഗിച്ചതിനാൽ systems smaller, faster, cheaper ആയി. Fourth generation VLSI microprocessors ഉപയോഗിച്ച് personal computers popularise ചെയ്തു. Fifth generation artificial intelligence-ൽ focus ചെയ്യുന്നു. എല്ലാ generations-ലും functional units-ന്റെ basic von Neumann organisation ഒരുപോലെ നിലനിൽക്കുന്നു.

Previously Asked Questions

Q34 (4 marks, Apr 2025)

Explain the generations of computers.

- *Answer:*

Computer generations hardware technology-ഉം software capability-ഉം കാലക്രമത്തിൽ എങ്ങനെ മാറി എന്ന് describe ചെയ്യുന്നു.

1. **First Generation (1940-1956)** — Vacuum tubes switches/amplifiers ആയി ഉപയോഗിച്ചു. Magnetic drums storage ആയി ഉപയോഗിച്ചു. Machines വളരെ large, costly, power-hungry ആയിരുന്നു. പക്ഷേ mechanical devices-നെക്കാൾ complex calculations വേഗത്തിൽ ചെയ്യാൻ കഴിഞ്ഞു. Examples: **UNIVAC, ENIAC, EDVAC.**

2. **Second Generation (1956-1963)** — Vacuum tubes-നു പകരം **transistors** ഉപയോഗിച്ചു. Computers smaller, faster, cooler, more efficient ആയി. **COBOL, FORTRAN** പോലുള്ള high-level languages appeared. Examples: **IBM 1400 series, IBM 7090/7094, UNIVAC 1107, CDC 3600.**
3. **Third Generation (1964-1971)** — **Integrated Circuits (ICs)** ഉപയോഗിച്ചു. Semiconductor material-ൽ thousands of transistors ഉൾപ്പെടുത്തി. Systems more reliable, faster, smaller, cheaper ആയി; maintenance കുറച്ചു. Examples: **IBM 370, PDP-11, IBM System/360, UNIVAC 1108, Honeywell-6000.**
4. **Fourth Generation (1971-2010)** — **VLSI / microprocessors** അടിസ്ഥാനമാക്കി. One silicon chip-ൽ many ICs ഉണ്ടായി. Personal computers, laptops, handheld devices widely spread ചെയ്തു. Examples: **Apple, CRAY-1.**
5. **Fifth Generation (present and future)** — **Artificial Intelligence (AI)** അടിസ്ഥാനമാക്കി. Human thinking/acting mimic ചെയ്യാൻ ശ്രമിക്കുന്ന systems. Examples: **PARAM 10000, IBM notebooks.**

Diagram (refer SLM): Fig. 1.1.1 The evolution of computers.

1.1.2 Functional Units of a Computer

Theory

ഒരു computer input സ്വീകരിക്കുകയും, internally stored programs ഉപയോഗിച്ച് അത് process ചെയ്യുകയും, results output devices-ലേക്ക് നൽകുകയും ചെയ്യുന്നു.

Computer Architecture എന്നത് program execution-നെ logical ആയി ബാധിക്കുന്ന attributes-നെ വിവരിക്കുന്നു. ഉദാഹരണങ്ങൾ:

- Instruction set
- Data representation
- I/O mechanisms
- Addressing

Computer Organisation എന്നത് ഈ architectural specifications നടപ്പിലാക്കുന്ന operational units-ഉം അവയുടെ interconnections-ഉം ആണ്.

Computer-ന്റെ പ്രധാന functional units:

1. Input Unit
2. Output Unit
3. Memory Unit

4. Arithmetic and Logic Unit (ALU)

5. Control Unit

ഈ units തമ്മിൽ **buses** എന്ന electrical cables ഉപയോഗിച്ച് communication നടത്തുന്നു.

CPU-യിൽ:

- Control Unit
- ALU
- Internal Registers

എന്നിവ ഉൾപ്പെടുന്നു.

Programs-ഉം data-യും input devices വഴി system-ലേക്ക് വരുന്നു. Execution സമയത്ത് അവ memory-യിൽ reside ചെയ്യുന്നു. Control Unit-ന്റെ supervision-ൽ ALU അവ process ചെയ്യുന്നു. Results output devices-ലേക്ക് പോകുന്നു.

Important Points

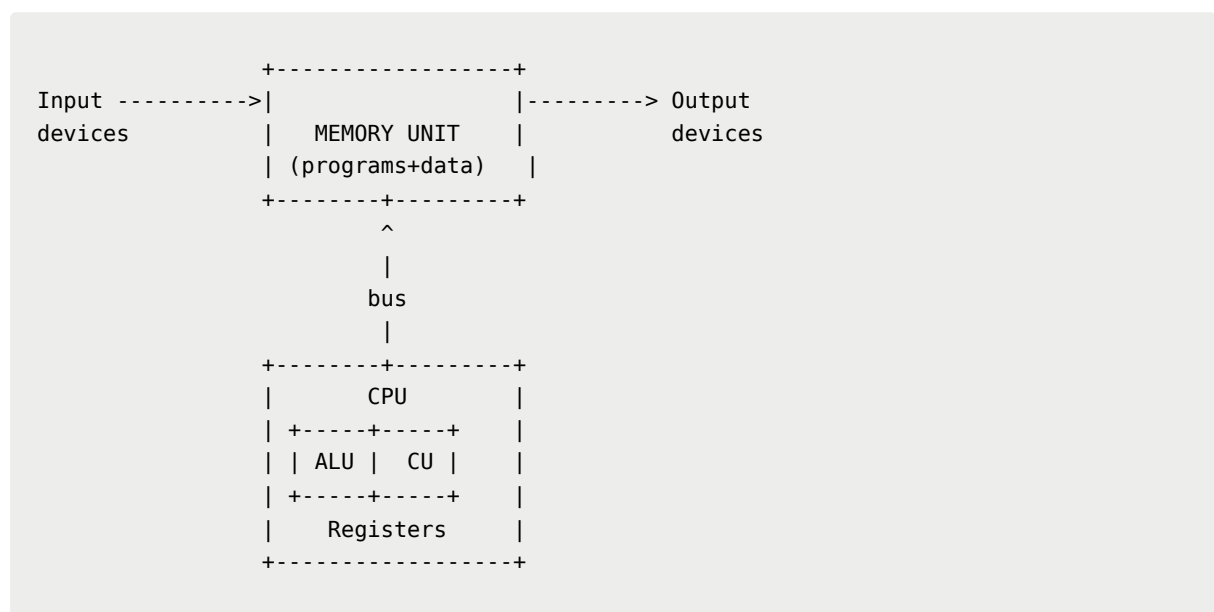
Five units:

Input | Output | Memory | ALU | Control

CPU = Control Unit + ALU + Registers

- Programs (code) and data രണ്ടും memory-യിൽ available ആയിരിക്കണം.
- Units തമ്മിലുള്ള communication **buses (shared wires)** ഉപയോഗിച്ചാണ്.

Diagram



Functional Units:

Input Unit → Memory Unit → Control Unit + ALU → Output Unit

For Exam

ഒരു basic computer-ന് അഞ്ച് functional units ഉണ്ട്. Input unit data-യും programs-ഉം system-ലേക്ക് കൊണ്ടുവരുന്നു. Memory instructions, data, results എന്നിവ store ചെയ്യുന്നു. ALU arithmetic and logical operations perform ചെയ്യുന്നു. Control unit control signals issue ചെയ്ത് എല്ലാ activities-ഉം coordinate ചെയ്യുന്നു. Output unit results user-ന് present ചെയ്യുന്നു. CPU ALU, control unit, registers എന്നിവ ചേർന്നതാണ്. Data, address, control exchange ചെയ്യാൻ units buses വഴി linked ആയിരിക്കും.

Previously Asked Questions

Q11 (1 mark, Apr 2025)

Memory units stores _____ and _____.

- *Answer:*

Programs (code/instructions) and data

Computations-ന്റെ intermediate/final results-ഉം memory store ചെയ്യുന്നു.

Q36 (15 marks, SLM Model Set 1)

Explain the functional units of a computer according to the Neumann model. Discuss how each unit contributes to the overall functioning of a computer.

- *Answer:*

Introduction:

John von Neumann-ന്റെ **stored-program model** അനുസരിച്ച്, ഒരു computer-ന് buses വഴി ഒരുമിച്ച് പ്രവർത്തിക്കുന്ന അഞ്ച് പ്രധാന functional units ഉണ്ട്:

1. Input Unit
2. Output Unit
3. Memory Unit
4. Arithmetic and Logic Unit (ALU)
5. Control Unit

CPU = Control Unit + ALU + Registers

Programs-ഉം data-യും ഒരേ memory-യിൽ store ചെയ്യുകയും sequential control പ്രകാരം execute ചെയ്യുകയും ചെയ്യുന്നു.

1. Input Unit

പുറത്തുനിന്നുള്ള data-യും programs-ഉം സ്വീകരിക്കുന്നു.

ഉദാഹരണങ്ങൾ:

- Keyboard
- Mouse
- Scanner
- Disk

ഇവ processor-നും memory-ക്കും അനുയോജ്യമായ **binary form**-ലേക്ക് convert ചെയ്യുന്നു.

Input ഇല്ലെങ്കിൽ system-ന് process ചെയ്യാൻ ഒന്നുമുണ്ടാകില്ല.

2. Memory Unit

താഴെ പറയുന്നവ store ചെയ്യുന്നു:

- Execute ചെയ്യേണ്ട instructions
- Process ചെയ്യേണ്ട data
- Intermediate results
- Final results

Memory-ക്ക് രണ്ട് പ്രധാന classes ഉണ്ട്:

- **Primary Memory** — fast ROM/RAM
- **Secondary Memory** — disks പോലുള്ള bulk permanent storage

Execution സമയത്ത് programs primary memory-യിൽ ഉണ്ടായിരിക്കണം.

Memory CPU-ന് data/instructions നൽകുകയും പിന്നീട് output-നായി results സ്വീകരിക്കുകയും ചെയ്യുന്നു.

3. Arithmetic and Logic Unit (ALU)

ALU arithmetic operations നടത്തുന്നു:

- -
- -

- ×
- ÷

Logical/comparison operations-ഉം നടത്തുന്നു:

- Equal
- Less than
- Greater than

ALU CPU-യുടെ **computational work area** ആണ്.

Results registers-ലേക്കോ memory-യിലേക്കോ തിരികെ എഴുതാം.

4. Control Unit

Control Unit computer-ന്റെ **nerve centre** ആണ്.

ഇത്:

- Instructions fetch ചെയ്യുന്നു.
- Instructions interpret ചെയ്യുന്നു.
- Control signals generate ചെയ്യുന്നു.
- Timing നൽകുന്നു.
- Input, memory, ALU, output എന്നിവയിലെ data transfers coordinate ചെയ്യുന്നു.
- Instruction cycle-ന്റെ ഓരോ ഘട്ടത്തിലും എന്ത് ചെയ്യണമെന്ന് തീരുമാനിക്കുന്നു.

5. Output Unit

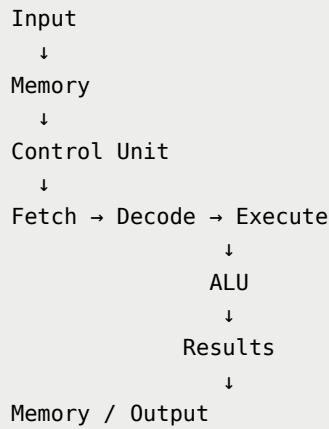
Processed results user-ന് നൽകുന്നു.

ഉദാഹരണങ്ങൾ:

- Monitor
- Printer
- Plotter
- Speaker

Internal binary results മനുഷ്യന് ഉപയോഗിക്കാൻ കഴിയുന്ന രൂപത്തിലേക്ക് convert ചെയ്യുന്നു.

How They Work Together



Input programs/data memory-യിൽ കൊണ്ടുവരുന്നു.

Control Unit **fetch** → **decode** → **execute** sequence നിയന്ത്രിക്കുന്നു.

ALU operands process ചെയ്യുന്നു.

Results memory-യിലേക്ക് write back ചെയ്യുകയും output-ലേക്ക് അയക്കുകയും ചെയ്യുന്നു.

Buses units തമ്മിൽ:

- Data
- Addresses
- Control signals

carry ചെയ്യുന്നു.

ഓരോ unit-നും പ്രത്യേക role ഉണ്ടെങ്കിലും, അവയുടെ coordinated operation ആണ് complete computing system സാധ്യമാക്കുന്നത്.

Exam-ൽ von Neumann block diagram വരയ്ക്കുക.

CPU = **ALU + CU + Registers**

Diagram (refer SLM): Fig. 1.1.2 Functional Units of a Computer.

1.1.2.1 Input Unit

Theory

CPU/memory-യിൽ എത്തുന്നതിന് മുമ്പ് external input-നെ **binary form**-ലേക്ക് convert ചെയ്യുന്നതാണ് Input Unit.

ഇത് താഴെ പറയുന്ന sources-ൽ നിന്ന് data/programs സ്വീകരിക്കാൻ access നൽകുന്നു:

- Keyboard

- Mouse
- Disks
- Scanners
- Cameras

Important Points

Keyboard: ഓരോ key press-നും controller CPU/memory-ലേക്ക് ഒരു code അയയ്ക്കുന്നു.

Mouse / Trackball / Touchpad:

- Menus select ചെയ്യാൻ
- Draw/paint ചെയ്യാൻ

Joystick: Games-നായി ഉപയോഗിക്കുന്നു.

Scanners and Cameras: Images-നെ digitise ചെയ്യുന്നു.

Input-ൽ നിന്നുള്ള encoded information processor-ലേക്ക് അയക്കുന്നു.

For Exam

Input unit human/machine input-നെ binary impulses ആയി convert ചെയ്ത് data and programs computer-ലേക്ക് supply ചെയ്യുന്നു. Common devices: keyboard, mouse, joystick, scanner, camera, trackball, touchpad. Input magnetic/optical disks-ൽ നിന്നുമാകാം.

1.1.2.2 Memory Unit

Theory

Memory Unit താഴെ പറയുന്നവ store ചെയ്യുന്നു:

- Execute ചെയ്യേണ്ട program instructions
- Process ചെയ്യേണ്ട data
- Computations-ന്റെ results

Memory-ക്ക് രണ്ട് classes ഉണ്ട്:

1. **Primary Memory**
2. **Secondary Memory**

Important Points

Type	Nature	Role	Examples
ROM (Primary)	Non-volatile	Firmware: BIOS, POST, I/O drivers; read-only system programs	ROM
RAM (Primary)	Volatile, expensive, fast	Run-time instructions and data	RAM
Secondary	Non-volatile, cheaper, slower	Bulk/Auxiliary storage	Hard disk, optical disk, semiconductor storage

ROM

- Non-volatile ആണ്.
- Firmware store ചെയ്യുന്നു.
- Examples: BIOS, POST, I/O drivers.
- Read-only system programs store ചെയ്യുന്നു.

RAM

- Volatile ആണ്.
- Fast ആണ്.
- Expensive ആണ്.
- Run-time instructions and data store ചെയ്യുന്നു.
- User/read-write memory ആണ്.

Secondary Memory

- Non-volatile
- Cheaper
- Slower
- Bulk/Auxiliary storage

Examples:

- Hard disk
- Optical disk
- Semiconductor storage

Secondary memory-യുടെ access time > Primary memory-യുടെ access time.

Execution സമയത്ത് electronic-speed processing-നായി programs main memory-യിൽ ഉണ്ടായിരിക്കണം.

For Exam

Memory code, data, results എന്നിവ store ചെയ്യുന്നു. Primary memory (ROM + RAM) fast semiconductor storage ആണ്. ROM non-volatile firmware storage ആണ്. RAM volatile run-time memory ആണ്. Secondary memory cheaper, permanent, slower ആണ് (disks etc.) കൂടാതെ primary storage-നെ supplement ചെയ്യുന്നു.

Previously Asked Questions

Q26 (4 marks, SLM Model Set 1)

Explain the difference between primary and secondary memory.

- *Answer:*

Point	Primary Memory	Secondary Memory
Nature	Fast semiconductor storage; CPU-ന് directly accessible	Disks, optical media തുടങ്ങിയ bulk/auxiliary storage
Volatility	RAM volatile; ROM non-volatile firmware	Generally non-volatile
Speed / Cost	Faster; per bit കൂടുതൽ expensive	Slower; large capacity-യ്ക്ക് cheaper
Role	Currently running programs, data, intermediate results	Permanent/long-term programs and files
Examples	ROM, RAM	Hard disk, optical disk, semiconductor secondary storage

During Execution

Program electronic-speed processing-നായി **primary/main memory-യിൽ** ഉണ്ടായിരിക്കണം.

Secondary memory backing store ആയി പ്രവർത്തിക്കുന്നു. ആവശ്യമുള്ളപ്പോൾ contents primary memory-യിലേക്ക് load ചെയ്യാം.

Summary

Primary memory: Fast, limited, often volatile working memory.

Secondary memory: Large, permanent, slower storage.

Secondary memory-യുടെ access time primary memory-നേക്കാൾ കൂടുതലാണ്.

1.1.2.3 Arithmetic and Logic Unit (ALU)

Theory

ALU CPU-വിനുള്ളിലെ ഒരു **digital circuitry** ആണ്.

ഇത് arithmetic operations നടത്തുന്നു:

- Add
- Subtract
- Multiply
- Divide

Logical operations-ഉം നടത്തുന്നു:

- Equal
- Less than
- Greater than

ഇതിനായി **adders** and **comparators** പോലുള്ള circuits ഉപയോഗിക്കുന്നു.

ALU CPU-യുടെ ഒരു indispensable building block ആണ്.

Important Points

Arithmetic Unit:

- $+$, $-$, $\times$, $\div$

Logic Unit:

Comparisons and logical decisions

ALU:

Mathematical/logical data manipulation-ന്റെ work area.

For Exam

ALU data-യിൽ arithmetic and logical operations perform ചെയ്യുന്നു. Adder, comparator, related circuits എന്നിവ ഇതിൽ അടങ്ങിയിരിക്കുന്നു. ഇത് CPU-യുടെ core part ആണ്.

Previously Asked Questions

Q16 (2 marks, SLM Model Set 1)

Explain the role of the Arithmetic and Logic Unit (ALU) in a computer.

- *Answer:*

ALU CPU-വിനുള്ളിലെ digital circuitry ആണ്.

ഇത് എല്ലാ arithmetic operations-ഉം നടത്തുന്നു:

- Addition
- Subtraction
- Multiplication
- Division

Logical operations-ഉം നടത്തുന്നു:

- Equal
- Less than
- Greater than
- Related logic

ALU **adders and comparators** പോലുള്ള circuits ഉപയോഗിക്കുന്നു.

ഇത് processor-ന്റെ computational work area ആയി പ്രവർത്തിക്കുന്നു.

Control Unit-ന്റെ supervision-ൽ registers/memory-യിൽ നിന്നുള്ള operands ALU process ചെയ്യുന്നു.

Results registers-ലേക്കോ memory-യിലേക്കോ write back ചെയ്യുന്നു.

1.1.2.4 Control Unit

Theory

Control Unit computer-ന്റെ **nerve centre** ആണ്.

ഇത് താഴെ പറയുന്നവയുടെ activities coordinate ചെയ്യുന്നു:

- Input
- Output
- ALU
- Memory

Instructions interpret ചെയ്യുകയും data transfers നിയന്ത്രിക്കുന്ന **control signals** issue ചെയ്യുകയും operations-ന് timing നൽകുകയും ചെയ്യുന്നു.

Important Points

- എല്ലാ units-ന്റെയും overall coordination
- Data transfer-നും timing-നും control signals നൽകുന്നു
- ഓരോ instance-ലും എന്ത് operation/action വേണമെന്ന് തീരുമാനിക്കുന്നു
- ALU + Registers എന്നിവയോടൊപ്പം CPU രൂപീകരിക്കുന്നു

For Exam

Control unit nerve centre ആണ്. ഇത് instructions interpret ചെയ്യുകയും correct timing-യോടെ input, output, memory, ALU operations coordinate ചെയ്യാൻ control signals generate ചെയ്യുകയും ചെയ്യുന്നു.

Previously Asked Questions

Q1 (1 mark, SLM Model Set 1)

What is the primary function of the Control Unit in a computer?

- *Answer:*

Computer-ന്റെ എല്ലാ units-ഉം **coordinate/control** ചെയ്യുക.

Instructions interpret ചെയ്ത് data transfers-നും operations-നും ആവശ്യമായ **control signals and timing** issue ചെയ്യുന്നു.

Q1 (1 mark, SLM Model Set 2)

What are the two types of control signals generated by the control unit?

- *Answer:*

1. **Timing signals**
2. **Command signals**

1.1.2.5 Output Unit

Theory

Processing കഴിഞ്ഞ ശേഷം computer results-ഉം messages-ഉം **Output Unit** വഴി നൽകുന്നു.

Standard output device:

Monitor

Monitor types:

- CRT
- LCD/TFT
- LED

Other output devices:

- Printers
- Projectors
- Plotters
- Speakers

Important Points

Monitor types: CRT, LCD/TFT, LED

Other outputs: Printer, Projector, Plotter, Speaker

For Exam

Output unit computation results user-ന് monitors, printers, plotters, projectors, speakers തുടങ്ങിയ devices വഴി present ചെയ്യുന്നു.

1.1.3 Basic Operational Concepts

Theory

ഒരു instruction-ന് രണ്ട് പ്രധാന parts ഉണ്ട്:

1. Opcode

Perform ചെയ്യേണ്ട operation.

Examples:

- ADD
- LOAD
- STORE

2. Operand(s)

Operation-ൽ ഉപയോഗിക്കുന്ന:

- Data
- Register
- Memory address

Example

```
ADD A, 5
```

ഇവിടെ:

ADD = Opcode

A, 5 = Operands

Steps for ADD (Address of D), R0

1. Main memory-യിൽ നിന്ന് instruction processor-ലേക്ക് fetch ചെയ്യുക.
2. Memory-യിലെ address D-ൽ ഉള്ള operand fetch ചെയ്യുക.
3. Memory operand-നെ R0-യുടെ contents-ലേക്ക് add ചെയ്യുക.
4. Sum R0-ൽ store ചെയ്യുക.

Overall Computer Operation

Program main memory-യിൽ reside ചെയ്യുന്നു.

↓

CPU instruction fetch/decode/execute ചെയ്യുന്നു.

↓

ALU operations process ചെയ്യുന്നു.

↓

Results output-ലേക്ക് പോകുന്നു.

↓

Control Unit എല്ലാ activities-ഉം monitor ചെയ്യുന്നു.

Important Points

Opcode = What to do

Operand = Data / Location

Instruction cycle-ൽ:

- Fetch
- Decode
- Execute
- Store

എന്നിവ ഉൾപ്പെടുന്നു.

Program Counter (PC) next instruction track ചെയ്യുന്നു.

Control Unit sequencing manage ചെയ്യുന്നു.

For Exam

Instructions opcode and operands contain ചെയ്യുന്നു. CPU memory-യിൽ നിന്ന് instructions fetch ചെയ്യുന്നു, decode ചെയ്യുന്നു, ALU-ൽ operations execute ചെയ്യുന്നു, results store ചെയ്യുന്നു. ഈ എല്ലാം control-unit supervision-ൽ നടക്കുന്നു.

Previously Asked Questions

Q19 (2 marks, SLM Model Set 1)

Mention the various phases in executing an instruction.

- *Answer:*

Instruction cycle-ന്റെ പ്രധാന phases:

1. **Fetch** — memory-യിൽ നിന്ന് instruction processor-ലേക്ക് കൊണ്ടുവരുന്നു.
2. **Decode** — opcode-ഉം addressing information-ഉം interpret ചെയ്യുന്നു.
3. **Fetch operands** — ആവശ്യമായാൽ registers/memory-യിൽ നിന്ന് operands എടുക്കുന്നു.
4. **Execute** — ALU അല്ലെങ്കിൽ transfer unit operation നടത്തുന്നു.
5. **Store / Write-back** — result register അല്ലെങ്കിൽ memory-യിലേക്ക് എഴുതുന്നു.
6. **Interrupt check** — pending interrupts ഉണ്ടെങ്കിൽ അടുത്ത fetch-ന് മുമ്പ് handle ചെയ്യുന്നു.

Short form — Exam

Fetch → Decode → Execute → Store

1.1.4–1.1.5 Bus Structures and Bus Types

Theory

Bus എന്നത് multiple devices തമ്മിൽ communication നടത്തുന്നതിനുള്ള ഒരു group of wires / shared transmission medium ആണ്.

Bus width ഒരേസമയം എത്ര bits transfer ചെയ്യാമെന്ന് നിർണ്ണയിക്കുന്നു.

Example:

16-bit address bus

$$2^{16} = 64K \text{ locations}$$

ഓരോ unit-നും തമ്മിൽ separate wires ഉപയോഗിക്കുന്നതിന് പകരം **common bus** ഉപയോഗിക്കാം.

Types of Bus

1. Internal Bus / System Bus

CPU-യെ:

- Internal circuitry
- Memory
- I/O units

എന്നിവയുമായി connect ചെയ്യുന്നു.

2. External Bus

External peripherals-നെ CPU-യുമായി connect ചെയ്യുന്നു.

Examples:

- Keyboard
- Mouse
- Scanner

System Bus — Three Groups

Bus	Direction	Function
Data Bus	Bi-directional	CPU, memory, ports എന്നിവയ്ക്കിടയിൽ data carry ചെയ്യുന്നു
Address Bus	Unidirectional	Source/destination address carry ചെയ്യുന്നു
Control Bus	Control signals	Read/write, interrupt, timing, coordination

Sending Data

Obtain bus control → Place address → Place data → Send control signals → Wait for acknowledgement

Requesting Data

Obtain bus → Place address → Send read → Wait → Receive data from data bus

Single-Bus Structure

ഒരു common bus:

CPU + Memory + I/O

എന്നിവയെ connect ചെയ്യുന്നു.

I/O devices connect ചെയ്യുന്നതിനുള്ള usual organisation ആണ് **single-bus structure**.

Dedicated vs Multiplexed

Dedicated

Lines ഒരു particular function-ന് permanently assigned ആയിരിക്കും.

ഉദാഹരണം:

Separate address lines + separate data lines.

Multiplexed

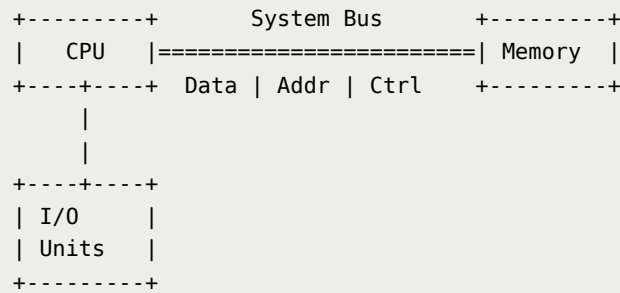
Same lines different purposes-ന് different times-ൽ ഉപയോഗിക്കുന്നു.

ഉദാഹരണം:

Address/Data multiplexing.

ഇത് **time multiplexing** ആണ്.

Diagram



For Exam

Bus CPU, memory, peripherals എന്നിവ തമ്മിലുള്ള communication-നുള്ള shared set of lines ആണ്. System bus-ൽ bi-directional data bus, unidirectional address bus, control bus എന്നിവ ഉണ്ട്. Internal buses CPU internals connect ചെയ്യുന്നു; external buses peripherals connect ചെയ്യുന്നു. Bus lines dedicated അല്ലെങ്കിൽ multiplexed ആയിരിക്കാം. I/O devices connect ചെയ്യാൻ common ആയി single/common bus structure ഉപയോഗിക്കുന്നു.

Previously Asked Questions

Q19 (2 marks, Apr 2025)

Explain different bus types.

- *Answer:*

Buses units തമ്മിൽ:

- Data
- Address
- Control information

exchange ചെയ്യുന്നു.

Internal/System Bus

Processor, memory, I/O എന്നിവ connect ചെയ്യുന്നു.

External Bus

Keyboard, mouse തുടങ്ങിയ peripherals CPU-യുമായി connect ചെയ്യുന്നു.

System Bus Types

Data Bus: Bi-directional data path.

Address Bus: Unidirectional addresses.

Control Bus: Read/write and other control signals.

Bus lines:

- Dedicated
- Multiplexed

ആയിരിക്കാം.

Q4 (1 mark, Apr 2025)

What is the usual BUS structure used to connect the I/O devices?

- *Answer:*

Single bus structure

അഥവാ CPU, memory, I/O എന്നിവ connect ചെയ്യുന്ന **common/single bus**.

Diagram (refer SLM): Fig. 1.1.3 System Bus.

Unit 2: Timing and Control

1.2.1 Building Blocks — Flip-flops, Registers, Decoder, Latches, Clock

Theory

Instruction execution എന്നത് cycles-ന്റെ ഒരു sequence ആണ്.

Fetch → Decode → Execute → Store

ഓരോ cycle-ലും multiple **micro-operations** ഉണ്ടാകും.

Examples:

- Register transfers
- ALU operations
- Bus transfers

Control Unit-ന്റെ രണ്ട് primary functions:

1. **Sequencing** — micro-operations ശരിയായ order-ൽ step-by-step നടത്തുക.

2. Execution — ഓരോ micro-operation-ഉം നടത്താൻ ആവശ്യമായ control signals നൽകുക.

Flip-flops

Flip-flop ഒരു **bistable device** ആണ്.

ഇത് ഒരു bit:

0 അല്ലെങ്കിൽ 1

store ചെയ്യുന്നു.

Uses:

- Registers
- Counters
- SRAM cells
- State machines

Types:

- SR
 - D
 - JK
 - T
-

Registers

Registers CPU-വിനുള്ളിലെ **small, high-speed storage** ആണ്.

ഇവ store ചെയ്യുന്നത്:

- Data
- Addresses
- Instructions
- Status

RAM-നേക്കാൾ registers faster ആണ്, കാരണം അവ CPU-വിനുള്ളിലാണ്.

Register and Use

Register	Use
ACC	Intermediate arithmetic/logic results
PC	Address of next instruction
IR	Current instruction being decoded/executed
DR	Temporary data to/from memory or I/O
MAR	Address of memory location to access
MBR	Data/instruction buffer with memory
SP	Top of stack
Index / Status / Control	Indexed addressing; flags; CPU control

Decoder

Encoded inputs-നെ specific control signals / one-of-n lines ആയി translate ചെയ്യുന്നു.

Uses:

- Instruction decoding
- Address decoding
- Control-signal generation

Types:

- Binary decoder
- BCD decoder
- Address decoder
- Instruction decoder

Example:

2-to-4 binary decoder

Latches

Latch ഒരു **level-sensitive bistable storage** ആണ്.

Types:

- SR

- D
- JK
- T

Flip-flops-ന്റെയും timing circuits-ന്റെയും building blocks ആയി ഉപയോഗിക്കുന്നു.

Clock

Clock digital devices synchronize ചെയ്യുന്നതിനുള്ള precise pulses നൽകുന്ന circuit ആണ്.

Clock cycle = consecutive clock pulses-ന്റെ corresponding edges-ക്കിടയിലെ interval.

Crystal oscillator clock frequency set ചെയ്യുന്നു.

Example:

100 MHz – 4 GHz

Secondary delayed clocks ഒരു cycle-ന്റെ ഉള്ളിൽ finer timing edges നൽകാം.

Important Points

Control Unit → Sequencing + Execution via control signals

Flip-flop = 1-bit storage

Register = Group of flip-flops

Decoder = Opcode/address → Select/control lines

Clock = Digital devices synchronization

For Exam

Timing and control flip-flops, registers, decoders, latches, master clock എന്നിവയിൽ depend ചെയ്യുന്നു. Clock എല്ലാ registers-ഉം synchronise ചെയ്യുന്നു. Decoders opcodes and sequence-counter outputs timing/control signals ആയി convert ചെയ്യുന്നു. Fast CPU operation-നായി registers addresses, data, instructions, status എന്നിവ hold ചെയ്യുന്നു.

Previously Asked Questions

Q1 (1 mark, Apr 2025)

What is used for synchronization of digital devices?

- *Answer:*

Clock / Clock pulses / Master clock generator

Q20 (2 marks, Apr 2025)

What is the use of decoder?

- *Answer:*

Decoder encoded binary inputs-നെ specific output line അല്ലെങ്കിൽ control signals ആയി translate ചെയ്യുന്നു.

CPU-യിൽ instruction **opcode** decode ചെയ്ത്:

- ALU
- Registers
- Buses

എന്നിവ control ചെയ്യാനുള്ള signals generate ചെയ്യുന്നു.

Address decoding-നും ഇത് ഉപയോഗിക്കുന്നു.

Sequence-counter outputs-നെ timing signals ആക്കാനും decoder ഉപയോഗിക്കുന്നു.

Example:

4-to-16 decoder → T0-T15

Q21 (2 marks, Apr 2025)

What is a register and mention the use of registers?

- *Answer:*

Register CPU-യ്ക്കുള്ളിലുള്ള ചെറിയതും വളരെ വേഗതയുള്ളതുമായ storage location ആണ്.

ഇത് flip-flops ഉപയോഗിച്ചാണ് നിർമ്മിക്കുന്നത്, binary data താൽക്കാലികമായി hold ചെയ്യുന്നു.

Registers ഉപയോഗിക്കുന്നത്:

- Intermediate results hold ചെയ്യാൻ — **Accumulator**
- അടുത്ത instruction-ന്റെ address hold ചെയ്യാൻ — **PC**
- നിലവിലെ instruction hold ചെയ്യാൻ — **IR**
- Memory address hold ചെയ്യാൻ — **MAR**
- Memory data buffer ചെയ്യാൻ — **MBR / DR**
- Stack top hold ചെയ്യാൻ — **SP**
- Index values, status flags എന്നിവ hold ചെയ്യാൻ

Registers main memory-നേക്കാൾ faster access നൽകുന്നു. Instruction execution-നും data transfer-നും registers essential ആണ്.

Q4 (1 mark, SLM Model Set 1)

Name the register typically used to hold an address for the memory unit.

- *Answer:*

MAR — Memory Address Register

Q2 (1 mark, SLM Model Set 2)

What does the Program Counter (PC) keep track of?

- *Answer:*

Program Counter (PC) memory-യിൽ നിന്ന് fetch ചെയ്യേണ്ട അടുത്ത instruction-ന്റെ address track ചെയ്യുന്നു. സാധാരണ sequential execution-ൽ ഓരോ fetch കഴിഞ്ഞും PC increment ചെയ്യും.

1.2.2 Timing and Control — Hardwired and Microprogrammed

Theory

Master clock generator എല്ലാ registers-ന്റെയും timing synchronize ചെയ്യുന്നു. Clock pulses മാത്രം register state change ചെയ്തില്ല; ബന്ധപ്പെട്ട **control signal enable** ആയിരിക്കുമ്പോഴാണ് register state മാറുന്നത്.

Control organisation രണ്ട് പ്രധാന തരങ്ങളാണ്:

1. **Hardwired control**
2. **Microprogrammed control**

1. Hardwired Control

Hardwired control-ൽ control logic gates, flip-flops, decoders, digital circuits എന്നിവ ഉപയോഗിച്ച് fixed ആയി നിർമ്മിച്ചിരിക്കും. ഇത് വളരെ **fast** ആണ്, പക്ഷേ modify ചെയ്യാൻ ബുദ്ധിമുട്ടാണ്, കാരണം change വേണമെങ്കിൽ logic rewiring/design change വേണം.

ഒരു basic hardwired Control Unit-ൽ സാധാരണ കാണുന്ന components:

- **Instruction Register (IR)** — instruction hold ചെയ്യുന്നു.
 - 1 bit — bit 15

- 3-bit opcode — bits 14-12
- Address bits — bits 0-11
- **3 × 8 decoder** — opcode decode ചെയ്ത് **D0-D7** lines ഉണ്ടാക്കുന്നു.
- **1-bit flip-flop** — indirect-addressing bit store ചെയ്യുന്നു.
- **4-bit sequence counter**
- **4 × 16 decoder** — timing signals **T0-T15** generate ചെയ്യുന്നു.
- **Control logic gates** — decoder outputs, timing signals, IR bits എന്നിവ combine ചെയ്ത് final control signals generate ചെയ്യുന്നു.

2. Microprogrammed Control

Microprogrammed control-ൽ control information **microinstructions** ആയി **control memory**-യിൽ store ചെയ്യുന്നു. Control memory സാധാരണയായി **ROM** ആണ്.

Design change ചെയ്യേണ്ടിവന്നാൽ hardware rewiring ചെയ്യുന്നതിനപകരം microprogram update ചെയ്യാൽ മതി. അതിനാൽ complex instructions handle ചെയ്യാൻ ഇത് easier ആണ്. പക്ഷേ hardwired control-നെക്കാൾ **slower** ആണ്.

Hardwired vs Microprogrammed

Hardwired Control	Microprogrammed Control
Logic circuits control signals generate ചെയ്യുന്നു	Microinstructions control signals generate ചെയ്യുന്നു
Faster	Slower
Modify ചെയ്യാൻ ബുദ്ധിമുട്ട്	Modify ചെയ്യാൻ എളുപ്പം
More expensive	More affordable
Complex instructions implement ചെയ്യാൻ ബുദ്ധിമുട്ട്	Complex instructions implement ചെയ്യാൻ എളുപ്പം
Limited instructions	Many instructions support ചെയ്യാം

For Exam

Timing and control unit instruction execution-നായി timing and control signals generate ചെയ്യുന്നു. Hardwired control digital circuits (IR, decoders, sequence counter, logic gates) ഉപയോഗിക്കുന്നു; ഇത് fast ആണ് പക്ഷേ inflexible ആണ്.

Microprogrammed control control words ROM-ൽ store ചെയ്യുന്നു; ഇത് slower ആണ് പക്ഷേ change ചെയ്യാൻ easier ആണ്. രണ്ടും master clock-ന്റെ കീഴിൽ registers, ALU, buses എന്നിവയുടെ micro-operations drive ചെയ്യുന്നു.

Diagram (refer SLM): Fig. 1.2.1 Clock and timing; Fig. 1.2.2 Hardwired Control Unit; Fig. 1.2.3 Microprogrammed Control; Table 1.2.1 comparison.

Previously Asked Questions

Q16 (2 marks, SLM Model Set 2)

What is the purpose of the Control Memory Address Register (CMAR) in a micro-programmed control unit?

- *Answer:*

CMAR — Control Memory Address Register control memory-യിലെ അടുത്ത microinstruction-ന്റെ address hold ചെയ്യുന്നു. ആ address-ിലുള്ള microinstruction fetch ചെയ്യുകയും അതിലെ control bits current micro-operation-നുള്ള control signals generate ചെയ്യുകയും ചെയ്യുന്നു.

അതിന് ശേഷം CMAR update ചെയ്യും:

- Next sequential address
- Branch address
- Opcode-ൽ നിന്ന് mapped address

ഇങ്ങനെ microprogram continue ചെയ്യാൻ CMAR സഹായിക്കുന്നു.

Q26 (4 marks, SLM Model Set 2)

What are the key components of a hardwired control unit? Briefly describe their functions.

- *Answer:*

Hardwired Control Unit fixed digital logic circuits ഉപയോഗിച്ച് control signals generate ചെയ്യുന്നു. പ്രധാന components:

1. **Instruction Register (IR)** — current instruction hold ചെയ്യുന്നു. I-bit, opcode, address fields എന്നിവ control logic-ന് നൽകുന്നു.
2. **Opcode decoder** — ഉദാ: **3 x 8 decoder**. Opcode decode ചെയ്ത് operation select lines **D0-D7** produce ചെയ്യുന്നു.
3. **I-bit flip-flop** — instruction-ലെ indirect-addressing bit store ചെയ്യുന്നു.
4. **Sequence Counter (SC)** — master clock അനുസരിച്ച് timing states വഴി മുന്നോട്ട് പോകുന്നു.

5. **Timing decoder** — ഉദാ: **4 x 16 decoder**. Timing signals **T0-T15** generate ചെയ്യുന്നു.
6. **Control logic gates** — decoder outputs, timing signals, IR bits എന്നിവ combine ചെയ്ത് registers, ALU, bus, memory എന്നിവയ്ക്കുള്ള final control signals ഉണ്ടാക്കുന്നു.
7. **Master clock** — sequence counter-നും register updates-നും synchronization നൽകുന്നു.

Hardwired control **fast** ആണ്, പക്ഷേ **inflexible** ആണ്. Design change ചെയ്യാൻ സാധാരണ logic rewiring വേണം.

1.2.3 Timing Signals and Sequence Counter

Theory

Sequence Counter (SC) clock pulses അനുസരിച്ച് increment ചെയ്യപ്പെടുന്നു. ഇതിലൂടെ successive timing states ഉണ്ടാകുന്നു. SC outputs decoder ഉപയോഗിച്ച് **T0, T1, T2, ...** എന്ന timing signals ആയി മാറുന്നു.

ഈ timing signals control logic ഉപയോഗിച്ച് instruction cycle-ലെ micro-operations ക്രമത്തിൽ നടത്തുന്നു.

Example:

T0 മുതൽ T4 വരെ timing signals മാത്രം ആവശ്യമെങ്കിൽ T4 കഴിഞ്ഞാൽ SC clear ചെയ്യാം.

D3T4: SC ← 0

ഇതിലൂടെ next cycle വീണ്ടും **T0** മുതൽ തുടങ്ങും.

SC positive-edge triggered ആണ്. Early clear ചെയ്യാത്ത പക്ഷം 4-bit SC **T5 ... T15** വരെ count ചെയ്ത് പിന്നെ T0-ലേക്ക് മടങ്ങും.

4-bit sequence counter 0 മുതൽ 15 വരെ count ചെയ്യും, അതായത് **0 to $2^4 - 1$** . 4-to-16 decoder ഉപയോഗിക്കുമ്പോൾ **16 timing signals (T0-T15)** generate ചെയ്യാൻ കഴിയും.

Important Points

- Timing signals instruction cycle-നുള്ളിലെ micro-operations sequence ചെയ്യുന്നു.
- Last needed timing state എത്തിയാൽ **CLR SC reset** ചെയ്യുന്നു.
- Decoder SC binary count-നെ one-hot timing lines ആയി convert ചെയ്യുന്നു.
- Clock മാത്രം മതിയല്ല; register changes enable ചെയ്യാൻ **control signals** വേണം.

For Exam

Timing signals sequence counter outputs decode ചെയ്താണ് produced ചെയ്യുന്നത്. അവ successive micro-operations enable ചെയ്യുന്നു. 4-bit sequence counter 0-15 വരെ count ചെയ്യുകയും 16 timing signals T0-T15 generate ചെയ്യുകയും ചെയ്യുന്നു.

Previously Asked Questions

Q2 (1 mark, Apr 2025)

How many timing signals could be generated by a 4 bit sequence counter?

- *Answer:*

16 timing signals.

Reason: 4-bit counter counts from **0 to $2^4 - 1 = 15$** , so outputs **T0 through T15**.

Diagram (refer SLM): Fig. 1.2.4 Example of Control Timing Signal (Mano).

Unit 3: Addressing Modes

1.3.1 Addressing Modes — Need and Types

Theory

ഒരു CPU support ചെയ്യുന്ന machine instructions-ന്റെ collection ആണ് **instruction set**. ഓരോ instruction-ലും:

- **Opcode** — ചെയ്യേണ്ട operation
- **Operand(s)** — operation ചെയ്യേണ്ട data

എന്നിവ ഉണ്ടാകും.

Operands immediate values ആയിരിക്കാം, CPU registers-ൽ ഉണ്ടായിരിക്കാം, അല്ലെങ്കിൽ memory locations-ൽ ഉണ്ടായിരിക്കാം.

Addressing modes എന്നത് operands എവിടെയാണ് കാണേണ്ടത്, എങ്ങനെ കണ്ടെത്തണം എന്നത് specify ചെയ്യുന്ന different formats/methods ആണ്. എല്ലാ computers-ലും ഒരേ addressing modes ഉണ്ടാകണമെന്നില്ല; architecture അനുസരിച്ച് modes മാറും.

Need for Addressing Modes

Addressing modes ആവശ്യമുള്ള കാരണങ്ങൾ:

- Memory addresses-ലേക്ക് pointers നൽകാൻ
- Loop control-നുള്ള counters support ചെയ്യാൻ

- Arrays / tables indexing ചെയ്യാൻ
- Program relocation support ചെയ്യാൻ
- Instruction-ലെ address field-ന് വേണ്ട bits കുറയ്ക്കാൻ
- Assembly programs കൂടുതൽ efficient and flexible ആയി എഴുതാൻ
- Fewer instructions / less execution time നേടാൻ

Instruction: [OPCODE][OPERAND / ADDRESS FIELD]

|
Operand എങ്ങനെ locate ചെയ്യും?
|
Addressing Mode

For Exam

Addressing modes ഒരു instruction-ന്റെ operand(s) CPU എങ്ങനെ find ചെയ്യുന്നു എന്ന് specify ചെയ്യുന്നു — instruction-പുറത്തുനിന്നോ, register-ലോ, memory-ൽ directly, indirectly, or indexing വഴിയോ. Flexible, compact, efficient programming-ന് ഇവ ആവശ്യമാണ്: pointers, loops, arrays, relocation, shorter address fields.

Previously Asked Questions

Q26 (4 marks, Apr 2025)

What is the need for addressing mode in computer architecture?

- *Answer:*

Addressing modes CPU instructions-ന്റെ operands എവിടെയാണ് ഉള്ളത് എന്ന് specify ചെയ്യുന്ന methods ആണ്. അവയുടെ need/advantages:

1. **Pointers** — fixed memory locations മാത്രം ഉപയോഗിക്കാതെ address pointers വഴി memory refer ചെയ്യാൻ.
2. **Loop control** — loops-ൽ data step-by-step access ചെയ്യാൻ counters support ചെയ്യാൻ.
3. **Indexing** — arrays/tables-ലെ elements base + index ഉപയോഗിച്ച് access ചെയ്യാൻ.
4. **Program relocation** — programs different memory starting addresses-ൽ run ചെയ്യാൻ സഹായിക്കാൻ.
5. **Shorter instructions** — registers അല്ലെങ്കിൽ relative forms ഉപയോഗിച്ച് address field bits കുറയ്ക്കാൻ.

6. **Efficiency and flexibility** — fewer instructions-ഉം better execution time-ഉം ഉള്ള assembly programs എഴുതാൻ.

Multiple addressing modes ഇല്ലെങ്കിൽ operand access rigid and inefficient ആകും.

Q27 (4 marks, SLM Model Set 1)

Define addressing modes and explain why they are important in computer architecture.

- *Answer:*

Addressing modes instruction operand(s) എവിടെയാണ് located എന്ന് specify ചെയ്യുന്ന different formats/methods ആണ് — instruction-പുറത്തെയോ, CPU register-ലോ, memory-ലോ direct/indirect/indexed രീതിയിലോ.

Importance:

1. Memory address pointers നൽകുന്നു.
2. Loop control counters support ചെയ്യുന്നു.
3. Arrays and tables indexing ചെയ്യാൻ സഹായിക്കുന്നു.
4. Program relocation support ചെയ്യുന്നു.
5. Address field-ന് വേണ്ട bits കുറയ്ക്കുന്നു.
6. Efficient and flexible assembly programming അനുവദിക്കുന്നു.

Implied (Implicit) Mode

Theory

ഈ mode-ൽ operands instruction definition-ൽ തന്നെ **implicitly** specified ആയിരിക്കും. പലപ്പോഴും accumulator ആണ് operand. Zero-address stack instructions-ഉം implied mode-ൽ ഉൾപ്പെടുന്നു.

Instruction-ൽ opcode മാത്രം ഉണ്ടായിരിക്കാം; operand field ഇല്ലായിരിക്കും.

Examples: CLC , INCA , DECA , NOP , RRC , RLC

Diagram (refer SLM): Fig. 1.3.1 Implied Addressing Mode.

Immediate Mode

Theory

Immediate mode-ൽ operand field-ൽ ഉപയോഗിക്കേണ്ട **actual data** തന്നെ ഉണ്ടായിരിക്കും. Registers initialize ചെയ്യാൻ ഇത് convenient ആണ്. ഇത് true memory-addressing mode അല്ല.

Examples: `ADD 6` , `ADD 07` , `MOV AX, 40H` , `MVI B, 47` , `MOV AL, 30H` , `JMP 30021`

Diagram (refer SLM): Fig. 1.3.2 Immediate Mode.

Previously Asked Questions

Q2 (1 mark, SLM Model Set 1)

Which addressing mode directly includes the operand as part of the instruction?

- *Answer:*

Immediate addressing mode

Direct Addressing Mode

Theory

Direct addressing mode-ൽ instruction-ലെ address field memory-യിലെ operand-ന്റെ **address** contain ചെയ്യുന്നു.

Effective Address (EA) = instruction-ലെ address part

Branch instructions-ൽ address field branch target address ആയി പ്രവർത്തിക്കും.

Examples: `ADD AL, [0301]` , `LDA 2050` , `LHLD 3010` , `IN 45`

Diagram (refer SLM): Fig. 1.3.3 Direct Addressing Mode.

Register Mode

Theory

Register mode-ൽ operand CPU-യിലെ named **general-purpose register**-ൽ ഉണ്ടായിരിക്കും. Memory access ആവശ്യമില്ലാത്തതിനാൽ ഇത് fast ആണ്.

Examples: `MOV DX, TAX RATE` , `MOV COUNT, CX` , `MOV EAX, EBX` , `ADD R1, R2`

Diagram (refer SLM): Fig. 1.3.4 Register Addressing Mode.

Register Indirect Mode

Theory

Register indirect mode-ൽ instruction operand-ന്റെ effective address hold ചെയ്യുന്ന ഒരു register-നെ refer ചെയ്യുന്നു. Operand fetch ചെയ്യാൻ ഒരു memory reference വേണം. Full memory address instruction-ൽ എഴുതേണ്ടതില്ലാത്തതിനാൽ fewer bits മതിയാകും.

x86-style examples-ൽ offset often **BX, SP, SI, DI** registers-ൽ ഉണ്ടായിരിക്കും.

Examples: `MOV A, M`, `LDAX B`, `ADD R` → `Accumulator ← Accumulator + M[R]`

Diagram (refer SLM): Fig. 1.3.5 Register Indirect Addressing Mode.

Auto-increment / Auto-decrement Mode

Theory

ഇത് register indirect mode പോലെയാണ്. പക്ഷേ memory access-ന് മുമ്പ് അല്ലെങ്കിൽ ശേഷം address register automatically increment/decrement ചെയ്യും. Tables/arrays sequential ആയി access ചെയ്യാൻ ഇത് useful ആണ്.

Example: `Add R1, (R2)`

`R1 ← R1 + M[R2]`

Then `R2 ← R2 + d`

Effective address = register content, with automatic update.

Indirect Addressing Mode

Theory

Indirect addressing mode-ൽ instruction address field ഒരു memory location-ന്റെ address നൽകുന്നു. ആ memory location ആണ് operand-ന്റെ effective address hold ചെയ്യുന്നത്. ഇതിനെ pointer to pointer ആയി കാണാം.

ഇതിന് extra memory access ആവശ്യമാണ്.

Examples: `ADD @200H` → `AC ← AC + [[200H]]`, `LOAD R1, @1005`

Diagram (refer SLM): Fig. 1.3.6 Indirect Addressing Mode.

Indexed Addressing Mode

Theory

Indexed addressing mode-ൽ:

Effective Address = base/displacement + index register content

Symbolic form: `X(R)`

$$EA = X + (R)$$

Base fixed ആയി തുടരും; index register array scan ചെയ്യുമ്പോൾ change ചെയ്യും.

Examples: `Load R4, 4(R2)` , `MOV AX, [SI+05]` , `ADD AX, [BX+SI]`

Base = 2800H and index = 01H ആണെങ്കിൽ:

EA = 2801H

Diagram (refer SLM): Fig. 1.3.7 Indexed Addressing Mode.

Mode summary:

Implied	: operand opcode വഴി fixed ആണ്
Immediate	: data instruction-ൽ തന്നെ
Direct	: EA = instruction-ലെ address
Register	: operand named register-ൽ
Reg Indirect	: EA = register content
Auto +/-	: register indirect + automatic update
Indirect	: EA = M[instruction-ലെ address]
Indexed	: EA = base + index register

Previously Asked Questions

Q36 (15 marks, Apr 2025)

Explain different addressing modes with examples.

- *Answer:*

Addressing modes CPU instruction operands എവിടെ നിന്ന് obtain ചെയ്യണം എന്ന് specify ചെയ്യുന്ന methods ആണ്.

1. **Implied / Implicit mode:** Operand instruction-ൽ നിന്ന് തന്നെ understood ആണ്, often accumulator. Examples: `CLC` , `INCA` , `NOP` , `RRC` .
2. **Immediate mode:** Actual operand instruction-ൽ തന്നെ ഉണ്ടായിരിക്കും. Example: `MOV AX, 40H` , `ADD 07` .
3. **Direct addressing mode:** Instruction address field operand memory address contain ചെയ്യുന്നു. EA = address field. Example: `LDA 2050` .
4. **Register mode:** Operand named CPU register-ൽ ഉണ്ടായിരിക്കും. Example: `MOV EAX, EBX` , `ADD R1, R2` .
5. **Register indirect mode:** Register operand-ന്റെ effective address hold ചെയ്യുന്നു. Example: `MOV A, M` , `ADD R` .

6. **Auto-increment / Auto-decrement mode:** Register indirect പോലെ, പക്ഷേ address register automatically update ചെയ്യും. Sequential table access-ന് suitable.
7. **Indirect addressing mode:** Instruction address field effective address store ചെയ്ത memory location-നെ point ചെയ്യുന്നു. Example: `ADD @200H`.
8. **Indexed addressing mode:** $EA = \text{base address} + \text{index register content}$. Example: `MOV AX, [SI+05]`, `Load R4, 4(R2)`.

Addressing modes pointers, loops, indexing, relocation, shorter address fields, efficient programming എന്നിവ support ചെയ്യുന്നു.

Q35 (4 marks, SLM Model Set 1)

Explain the difference between Direct and Indirect Addressing Modes.

- *Answer:*

Point	Direct Addressing	Indirect Addressing
Meaning	Instruction address field operand-ന്റെ memory address contain ചെയ്യുന്നു	Instruction address field effective address hold ചെയ്യുന്ന memory location-നെ point ചെയ്യുന്നു
Effective Address	$EA = \text{instruction-address}$	$EA = M[\text{address field}]$
Memory references	Operand fetch ചെയ്യാൻ സാധാരണ one memory access	EA കിട്ടാൻ extra memory access വേണം
Speed	Faster	Slower
Example	<code>LDA 2050</code> , <code>ADD AL, [0301]</code>	<code>ADD @200H</code> , <code>LOAD R1, @1005</code>

Summary: Direct mode instruction നേരിട്ട് operand address-നെ point ചെയ്യുന്നു. Indirect mode operand address store ചെയ്ത location-നെ point ചെയ്യുന്നു. Indirect mode കൂടുതൽ flexible ആണ്, പക്ഷേ slower ആണ്.

Q35 (4 marks, SLM Model Set 2)

What is the significance of Indexed Addressing Mode, and how does it function?

- *Answer:*

Indexed addressing-ൽ effective address:

$EA = \text{base/displacement} + \text{index register contents}$

Instruction fixed base/displacement നൽകുകയും index register specify ചെയ്യുകയും ചെയ്യുന്നു. CPU ഇവ രണ്ടും add ചെയ്ത് operand-ന്റെ memory address കണ്ടെത്തുന്നു.

Significance: Arrays, tables, repetitive data access എന്നിവയ്ക്ക് indexed addressing വളരെ useful ആണ്. Base fixed ആയി നിൽക്കും; loop-ൽ index change ചെയ്ത് successive elements access ചെയ്യാം.

Example: `Load R4, 4(R2)`, `MOV AX, [SI+05]`

Base = 2800H, index = 01H ആണെങ്കിൽ EA = 2801H.

Unit 4: Program Control

1.4.1 Program Control Instructions — Overview

Theory

Programs consecutive memory locations-ൽ store ചെയ്യപ്പെടുന്നു. **Program Counter (PC)** അടുത്ത instruction-ന്റെ address hold ചെയ്യുന്നു. സാധാരണ ഓരോ fetch കഴിഞ്ഞും PC increment ചെയ്യും, അതിനാൽ execution sequential ആയിരിക്കും.

Instructions മൂന്ന് broad types ആയി കാണാം:

- **Data transfer**
- **Data manipulation**
- **Program control**

Program control instructions PC change ചെയ്ത് execution flow മാറ്റുന്നു. Branching, skipping, subroutine call, halting, interrupt handling എന്നിവ ഇതിൽ ഉൾപ്പെടുന്നു.

Sequential order വീട്ട് condition, loop, call, interrupt എന്നിവ handle ചെയ്യേണ്ടപ്പോൾ program control instructions ആവശ്യമാണ്.

Program Status Word (PSW) zero, carry, parity തുടങ്ങിയ condition flags hold ചെയ്യുന്നു. Conditional branches ഈ flags ഉപയോഗിക്കുന്നു.

Types of Program Control Instructions

1. Conditional branch / skip
2. Unconditional branch / skip / jump
3. Subroutines — `CALL / RET`
4. Halting — `NOP, HALT`

5. Interrupt instructions — RESET, TRAP, INTR

Important Points

- Data transfer/manipulation കഴിഞ്ഞാൽ control PC വഴി next fetch-ലേക്ക് മടങ്ങും.
- Branch/Jump/Skip conditional അല്ലെങ്കിൽ unconditional ആയിരിക്കാം.
- Skip instruction-ന് address field ഇല്ല; അത് next instruction skip ചെയ്യുന്നു.
- CMP flags set ചെയ്യുന്നു, arithmetic result store ചെയ്തില്ല.
- Conditional branch സാധാരണ compare കഴിഞ്ഞാണ് ഉപയോഗിക്കുന്നത്.

For Exam

Program control instructions PC change ചെയ്ത് normal sequential flow alter ചെയ്യുന്നു. Types include unconditional and conditional branch/skip/jump, subroutine call/return, halt/NOP, interrupt-related instructions. Conditional forms PSW flags ഉപയോഗിക്കുന്നു.

Previously Asked Questions

Q22 (2 marks, Apr 2025)

Mention the types of program control instructions.

- *Answer:*

Main types:

1. **Unconditional branch/jump/skip** — JMP, BR, SKP
2. **Conditional branch/skip** — JE, BNZ, SZA
3. **Subroutine instructions** — CALL, RET
4. **Halting instructions** — NOP, HALT
5. **Interrupt instructions** — RESET, TRAP, INTR

Diagram (refer SLM): Fig. 1.4.1 Program Status Word; Table 1.4.1-1.4.2 program control / conditional branch examples.

1.4.2 Unconditional Branch Instruction

Theory

Unconditional branch flags test ചെയ്യാതെ തന്നെ control പുതിയ address-ലേക്ക് മാറ്റുന്നു. Branch/Jump സാധാരണ one-address instructions ആണ്.

Example:

BR ADR അല്ലെങ്കിൽ **JUMP ADR**

ഇത് ADR-നെ PC-ൽ load ചെയ്യുന്നു. അതിനാൽ next instruction target address-ൽ നിന്നാണ് fetch ചെയ്യുക.

Examples: **JUMP L2** , **BR ADR** , **SKP**

SKP execute ചെയ്താൽ next instruction execute ചെയ്യാതെ skip ചെയ്യും.

Previously Asked Questions

Q16 (2 marks, Apr 2025)

Explain unconditional branch instructions.

- *Answer:*

Unconditional branch instructions condition test ചെയ്യാതെ program execution flow മാറ്റുന്നു. ഇവ new address PC-ൽ load ചെയ്യുകയോ next instruction skip ചെയ്യുകയോ ചെയ്യും. അതിനാൽ next fetch target location-ൽ നിന്നാണ് നടക്കുന്നത്.

Examples: **JMP** , **JUMP L2** , **BR ADR** , **SKP**

Conditional branch പോലെ flag check ഇല്ല; execute ചെയ്താൽ always branch/skip ചെയ്യും.

1.4.3-1.4.4 Compare and Conditional Branch

Theory

CMP R1, R2 comparison ചെയ്യാൻ subtract operation ഉപയോഗിക്കുന്നു, പക്ഷേ result store ചെയ്തില്ല. Instead, **PSW flags** update ചെയ്യും.

അടുത്ത conditional branch instruction ഈ flags test ചെയ്യും.

Condition true ആണെങ്കിൽ:

PC ← Effective Address

Condition false ആണെങ്കിൽ:

PC ← PC + 1

Examples:

- **JE address1**
- **BC address1**
- **SKI**

- SKO
- SPA
- SNA
- SZA
- SZE

Previously Asked Questions

Q25 (2 marks, SLM Model Set 1)

Explain the role of conditional branch instructions in program control.

- *Answer:*

Conditional branch instructions specified condition true ആണെങ്കിൽ മാത്രം PC change ചെയ്യുന്നു. Condition സാധാരണ PSW flags അടിസ്ഥാനമാക്കിയാണ് test ചെയ്യുന്നത്. Compare/arithmetic instruction flags set ചെയ്യും.

Condition true ആണെങ്കിൽ PC effective/branch address-ലേക്ക് മാറും. Condition false ആണെങ്കിൽ execution sequential ആയി continue ചെയ്യും.

ഇവ programs-ൽ decision making, if/else paths, loops, skips എന്നിവ implement ചെയ്യാൻ സഹായിക്കുന്നു.

Examples: JE , BNZ , BC , SZA

1.4.5 Subroutines

Theory

Subroutine ഒരു well-defined task perform ചെയ്യുന്ന program fragment ആണ്.

Calling program CALL instruction ഉപയോഗിച്ച് subroutine invoke ചെയ്യുന്നു. CALL control subroutine-ലേക്ക് transfer ചെയ്യും.

Subroutine end-ൽ RETURN / RET instruction control caller program-ലേക്ക്, call കഴിഞ്ഞുള്ള next instruction-ലേക്ക്, restore ചെയ്യും.

Previously Asked Questions

Q12 (1 mark, Apr 2025)

What is a subroutine?

- *Answer:*

Subroutine well-defined task perform ചെയ്യുന്ന self-contained program fragment ആണ്. മറ്റൊരു program ഇത് call ചെയ്യുന്നു. Task complete ആയാൽ **CALL** കഴിഞ്ഞുള്ള instruction-ലേക്ക് control return ചെയ്യും, സാധാരണ **RET** ഉപയോഗിച്ച്.

1.4.6-1.4.7 Halting and Interrupt Instructions

Theory

NOP — No Operation

NOP data operation ഒന്നും ചെയ്യില്ല. PC മാത്രം next instruction-ലേക്ക് advance ചെയ്യും. Timing delay, padding, alignment, debugging placeholder എന്നിവയ്ക്കായി ഉപയോഗിക്കുന്നു. NOP implied mode instruction ആണ്.

HALT

HALT processor-നെ idle/halted state-ലേക്ക് stop ചെയ്യുന്നു. Interrupt, reset, external action എന്നിവ വരുമ്പോൾ വരെ processor stop ആയിരിക്കും. Program end ചെയ്യാൻ പലപ്പോഴും HALT ഉപയോഗിക്കുന്നു.

Interrupt instructions normal execution താൽക്കാലികമായി suspend ചെയ്ത് I/O അല്ലെങ്കിൽ service routine handle ചെയ്യാൻ സഹായിക്കുന്ന mechanism ആണ്.

Instruction	Nature
RESET	Processor initialise ചെയ്യുന്നു / PC start address-ലേക്ക് മാറ്റുന്നു
TRAP	Non-maskable, highest priority, vectored
INTR	Level-triggered, maskable, lowest priority

Previously Asked Questions

Q3 (1 mark, SLM Model Set 1)

What happens when a HALT instruction is executed?

- *Answer:*

HALT instruction execute ചെയ്താൽ processor stops / idle state-ലേക്ക് പോകുന്നു. Interrupt, reset, external action എന്നിവ വന്നാൽ മാത്രമേ resume ചെയ്യൂ. Program end ചെയ്യാൻ ഇത് ഉപയോഗിക്കാം.

Q17 (2 marks, SLM Model Set 1)

Describe the purpose of the NOP instruction.

- *Answer:*

NOP — No Operation data operation ഒന്നും ചെയ്യില്ല. Program Counter next instruction-ലേക്ക് മാത്രം advance ചെയ്യും. Timing delays, padding, alignment, temporary debugging placeholders എന്നിവയ്ക്കായി ഇത് ഉപയോഗിക്കുന്നു. NOP implied-mode instruction ആണ്.

Q25 (2 marks, SLM Model Set 2)

Describe the purpose of the NOP instruction.

- *Answer:*

NOP data അല്ലെങ്കിൽ flags meaningful ആയി change ചെയ്യില്ല. PC next instruction-ലേക്ക് advance ചെയ്യും. Delay/timing, code alignment, temporary placeholder എന്നിവയ്ക്കായി ഉപയോഗിക്കുന്നു.

Block 2: I/O and DMA

Unit 1: Register Transfer Languages (RTL)

2.1.1-2.1.2 Introduction and Key Concepts

Theory

ഒരു digital system registers, decoders, ALU elements, control logic തുടങ്ങിയ modules കൊണ്ടാണ് നിർമ്മിക്കുന്നത്. ഇവ data paths-ഉം control paths-ഉം വഴി interconnected ആയിരിക്കും.

Register data-യിൽ നടക്കുന്ന elementary operations-നെ **microoperations** എന്ന് വിളിക്കുന്നു.

Register Transfer Language (RTL) registers തമ്മിലുള്ള transfers-ഉം operations-ഉം symbolic low-level notation ആയി describe ചെയ്യുന്നു. ഇത് internal organisation concise ആയി specify ചെയ്യാനും hardware design ചെയ്യാനും ഉപയോഗിക്കുന്നു.

Internal organisation define ചെയ്യുന്നത്:

1. Registers-ന്റെ set and functions
2. Microoperations-ന്റെ sequence
3. അവ initiate ചെയ്യുന്ന control

Key Concepts

Concept	Meaning	Example
Register	CPU-യിലെ fast flip-flop storage	R1, MAR, PC, IR
Transfer	Data ഒരു register-ൽ നിന്ന് മറ്റൊന്നിലേക്ക് copy ചെയ്യൽ	$R1 \leftarrow R2$
Operation	Registers-ൽ arithmetic/logic ചെയ്യൽ	$R3 \leftarrow R1 + R2$
Control signal	Active ആയാൽ transfer enable ചെയ്യുന്നു	if (Enable) then $R1 \leftarrow R2$
Conditional transfer	Condition true ആണെങ്കിൽ മാത്രം transfer	if (C=1) then $R1 \leftarrow R2$
Sequential / Parallel	One after another / same clock	$R1 \leftarrow R2$; $R3 \leftarrow R1$ vs $R2 \leftarrow R1$, $R1 \leftarrow R2$

Register Types

- **Accumulator**
- **General-purpose registers**
- **Special-purpose registers**
 - **MAR** — memory address
 - **MBR** — memory buffer
 - **PC** — next instruction address
 - **IR** — current instruction

Important Points

- Replacement operator $\leftarrow$ destination-ന് source value ലഭിക്കുന്നു എന്ന് സൂചിപ്പിക്കുന്നു; source unchanged ആണ്.
- Control function $P: R2 \leftarrow R1$ ആയി എഴുതുന്നു. $P = 1$ ആണെങ്കിൽ മാത്രം transfer നടക്കും.
- Clock edge assumed ആണ്; RTL statement-ൽ clock സാധാരണ എഴുതില്ല.
- Comma simultaneous microoperations separate ചെയ്യുന്നു.

Basic RTL Symbols

Symbol	Meaning	Example
Letters/numbers	Register names	R1, MAR, PC
$\leftarrow$	Transfer	$R2 \leftarrow R1$
()	Register part	$PC(0-7)$, $R2(L)$

Symbol	Meaning	Example
,	Simultaneous operations	$R2 \leftarrow R1, R1 \leftarrow R2$
+, -	Arithmetic	$R3 \leftarrow R1 + R2$
AND / OR / NOT / XOR	Logic	$R3 \leftarrow R1 \text{ OR } R2$
<<, >>	Shift	$R1 \leftarrow R1 \ll 1$
if ()	Condition	if (C=1) then $R1 \leftarrow R2$
BUS, M[addr]	Bus / memory	$R1 \leftarrow M[100]$

For Exam

RTL (Register Transfer Language) registers തമ്മിൽ data move/process ചെയ്യുന്ന microoperations-ന്റെ symbolic notation ആണ്. Example: $R3 \leftarrow R1 + R2$ R1 and R2 add ചെയ്ത് R3-ൽ store ചെയ്യുന്നു. Controlled transfer: $P: R2 \leftarrow R1$. RTL hardware design, logic synthesis, microarchitecture specification എന്നിവയിൽ ഉപയോഗിക്കുന്നു.

Previously Asked Questions

Q3 (1 mark, Apr 2025)

What does RTL stand for?

- Answer:

Register Transfer Language

Q3 (1 mark, SLM Model Set 2)

What does the term RTL stand for in digital circuits?

- Answer:

Register Transfer Language

Q27 (4 marks, SLM Model Set 2)

Describe how RTL can be used to represent the transfer of data between two registers with an example.

- Answer:

RTL register-to-register transfer replacement operator $\leftarrow$ ഉപയോഗിച്ച് represent ചെയ്യുന്നു.

Destination $\leftarrow$ Source

ഇതിന്റെ അർത്ഥം source register-ലെ contents destination register-ലേക്ക് copy ചെയ്യുന്നു. Source unchanged ആയിരിക്കും.

Example:

$R2 \leftarrow R1$

ഇത് R1-ലെ value R2-ലേക്ക് transfer ചെയ്യുന്നു.

Conditional transfer:

$P: R2 \leftarrow R1$

$P = 1$ ആണെങ്കിൽ മാത്രം transfer നടക്കും.

Same clock-ൽ simultaneous microoperations comma ഉപയോഗിച്ച് എഴുതുന്നു:

$R2 \leftarrow R1, R3 \leftarrow R4$

Diagram (refer SLM): Fig. 2.1.1 Block diagram of registers; Fig. 2.1.2 Transfer R1 to R2 when $P=1$; Table 2.1.1 symbols.

2.1.4 Bus and Memory Transfers; Micro-operations

Theory

Common bus multiplexers അല്ലെങ്കിൽ tri-state buffers ഉപയോഗിച്ച് shared lines-ൽ ഏത് register data drive ചെയ്യണമെന്ന് select ചെയ്യുന്നു.

k registers, ഓരോന്നും n bits ആണെങ്കിൽ:

n multiplexers of size $k \times 1$ ആവശ്യമാണ്.

Selection lines $S_1 S_0$ A/B/C/D registers-ൽ ഏതാണ് bus-ലേക്ക് വരേണ്ടത് എന്ന് select ചെയ്യുന്നു.

RTL notation:

$BUS \leftarrow C, R1 \leftarrow BUS$

Bus implied ആണെങ്കിൽ simply:

$R1 \leftarrow C$

Micro-operation Execution Cycle Stages

- Fetch
- Decode
- Fetch operands
- Execute
- Memory access

- Write-back
- Interrupt handling, if needed

Types of Micro-operations

- Data transfer
- Arithmetic
- Logical
- Shift

Fetch Example in RTL

```
MAR ← PC
MBR ← Memory[MAR]
IR ← MBR
PC ← PC + 1
```

For Exam

Common-bus RTL select controls ഉപയോഗിച്ച് registers shared pathways എങ്ങനെ ഉപയോഗിക്കുന്നു എന്ന് describe ചെയ്യുന്നു. Micro-operations instruction cycle compose ചെയ്യുന്ന elementary CPU steps ആണ്: transfer, arithmetic, logic, shift.

Previously Asked Questions

Q18 (2 marks, SLM Model Set 1)

Describe what happens during the Fetch Cycle of micro-operations.

- *Answer:*

Fetch cycle-ൽ CPU memory-യിൽ നിന്ന് next instruction processor-ലേക്ക് കൊണ്ടുവരുന്നു. Typical microoperations:

```
MAR ← PC
MBR ← Memory[MAR]
IR ← MBR
PC ← PC + 1
```

MAR ← PC — next instruction address address path-ൽ place ചെയ്യുന്നു.

MBR ← Memory[MAR] — instruction word memory-യിൽ നിന്ന് read ചെയ്യുന്നു.

$IR \leftarrow MBR$ — instruction IR-ൽ load ചെയ്യുന്നു.

$PC \leftarrow PC + 1$ — next instruction-ലേക്ക് PC point ചെയ്യുന്നു.

Q37 (15 marks, SLM Model Set 1)

Describe the architecture of a common bus system using multiplexers, explaining how data is transferred between multiple registers and how control signals determine the selected register.

• *Answer:*

ഓരോ register-നും മറ്റെല്ലാ registers-ലേക്കും dedicated wires കൊടുക്കുന്നത് costly and complex ആണ്. അതിനാൽ **common bus** ഉപയോഗിക്കുന്നു. Common bus selected register-ന്റെ data shared lines-ൽ drive ചെയ്യുകയും destination register load-enable signal വഴി data receive ചെയ്യുകയും ചെയ്യുന്നു.

Architecture:

- k registers, ഓരോന്നും n bits ആണെങ്കിൽ **n multiplexers** വേണം.
- ഓരോ multiplexer-ഉം **k x 1** size ആയിരിക്കും.
- ഓരോ register-ന്റെയും corresponding bit multiplexer input-ലേക്ക് പോകുന്നു.
- Common selection lines, ഉദാ: **S1 S0**, ഏത് register bus-ലേക്ക് വരണമെന്ന് select ചെയ്യുന്നു.
- 4 registers A, B, C, D ആണെങ്കിൽ **00, 01, 10, 11** ഇവ A/B/C/D select ചെയ്യും.
- Bus outputs destination registers-ന്റെ data inputs-ലേക്ക് പോകുന്നു.
- Tri-state buffers ഉപയോഗിച്ചും same bus idea implement ചെയ്യാം.

Data transfer C to R1:

1. Selection lines C bus-ലേക്ക് വരത്തക്കവണ്ണം set ചെയ്യുക: **BUS $\leftarrow$ C**
2. R1-ന്റെ load control assert ചെയ്യുക.
3. Clock edge-ൽ **R1 $\leftarrow$ BUS**

Only one source bus drive ചെയ്യണം. Load enabled destinations bus contents receive ചെയ്യും.

Memory transfers:

- MAR address hold ചെയ്യുന്നു.
- MBR data buffer ചെയ്യുന്നു.
- Read: address MAR-ൽ place ചെയ്ത് read issue ചെയ്യുന്നു; **MBR $\leftarrow$ Memory[MAR]**

- Write: address MAR-ൽ, data MBR-ൽ place ചെയ്ത് write issue ചെയ്യുന്നു.

Advantages:

- Fewer interconnection lines
- Flexible register-to-register and register-memory transfers
- Control select/load signals വഴി clear ആണ്

Limitation:

- ഒരു സമയത്ത് bus-ൽ one source transfer മാത്രമേ സാധിക്കൂ, unless multiple buses used.

Q17 (2 marks, SLM Model Set 2)

What is the role of a multiplexer in a common bus system?

- *Answer:*

Common bus system-ൽ multiplexer ഏത് register ആണ് bus drive ചെയ്യേണ്ടത് എന്ന് select ചെയ്യുന്നു. k registers, n bits ആണെങ്കിൽ n multiplexers of size k x 1 ഉപയോഗിക്കുന്നു. Selection lines source register select ചെയ്യുന്നു. Example: `BUS ← C`, then `R1 ← BUS`.

Diagram (refer SLM): Fig. 2.1.1 Block diagram of registers; related common-bus multiplexer figures in SLM.

Unit 2: Input-Output Organization

2.2.1-2.2.2 Peripheral Devices and ASCII

Theory

I/O subsystem central system-നും outside world-നും ഇടയിൽ communication നടത്തുന്നു. CPU വളരെ fast ആണ്. Keyboard പോലുള്ള slow input devices നേരിട്ട് CPU-നെ കാത്തിരിപ്പിച്ചാൽ processor idle ആകും. അതിനാൽ bulk data disks/tapes പോലുള്ള media-യിൽ prepare ചെയ്ത് high rate-ൽ transfer ചെയ്യുന്നു.

Computer-ന്റെ direct control-ൽ ഉള്ള devices-നെ **on-line devices** എന്ന് പറയുന്നു.

Computer-ലേക്ക് attached ആയ I/O devices-നെ **peripherals** എന്ന് പറയുന്നു.

Examples:

- Keyboard

- Display
- Printer
- Disk
- Tape

ASCII

ASCII — American Standard Code for Information Interchange

ASCII 7-bit code ആണ്. ഇതിൽ 128 characters represent ചെയ്യാം.

- 94 printable characters
- 34 control characters

Example:

Letter **A** = 1000001

Previously Asked Questions

Q14 (1 mark, Apr 2025)

Expansion of ASCII is

- *Answer:*

American Standard Code for Information Interchange

Q4 (1 mark, SLM Model Set 2)

How many bits are used in ASCII code?

- *Answer:*

Standard ASCII-ൽ **7 bits** ആണ് ഉപയോഗിക്കുന്നത്. അതിനാൽ 128 characters represent ചെയ്യാം. 8th bit parity അല്ലെങ്കിൽ extended ASCII-ൽ ഉപയോഗിക്കാം.

2.2.3 I/O Interface — Functions, Types, Components

Theory

CPU-യും peripheral devices-ഉം തമ്മിൽ technology, speed, data format, operating modes എന്നിവയിൽ വ്യത്യാസമുണ്ട്. ഈ വ്യത്യാസങ്ങൾ പരിഹരിക്കുന്നതിനായി **processor bus-നും ഓരോ peripheral-നും ഇടയിൽ Interface Unit** പ്രവർത്തിക്കുന്നു.

I/O Interface-ന്റെ Functions

1. **Data Transfer** — CPU-യും I/O devices-ഉം തമ്മിൽ data കൈമാറ്റം ചെയ്യുന്നു.
2. **Control Signal Management** — I/O പ്രവർത്തനങ്ങളെ coordinate ചെയ്യുന്നതിനായി control signals കൈകാര്യം ചെയ്യുന്നു.
3. **Data Conversion** — വ്യത്യസ്ത format-ുകളും signal-ുകളും തമ്മിൽ conversion നടത്തുന്നു.
4. **Synchronisation** — CPU-യുടെയും I/O device-ന്റെയും speed വ്യത്യാസം കാരണം data നഷ്ടപ്പെടുകയോ corrupt ആകുകയോ ചെയ്യാതിരിക്കാൻ timing synchronize ചെയ്യുന്നു.

Types — Overview

- Memory-mapped I/O
- Isolated (Port-mapped) I/O
- Programmed I/O (Polling)
- Interrupt-driven I/O
- DMA
- Synchronous / Asynchronous I/O
- Serial / Parallel I/O

Polling (Programmed I/O)

CPU device status flags വീണ്ടും വീണ്ടും check ചെയ്യുന്നു. Device ready ആകുമ്പോൾ data transfer നടത്തുന്നു.

Simple ആണ്, പക്ഷെ CPU time waste ചെയ്യുന്നു.

Components

- Data / Address / Control buses
- I/O ports
- Controllers
- Buffers / Latches
- Interrupt lines
- Clock / Timing
- Control logic

- Power management

For Exam

I/O interface CPU-യെയും peripherals-നെയും link ചെയ്യുന്നു. ഇത് data transfer, control, format conversion, synchronisation എന്നിവ handle ചെയ്യുന്നു. Interfaces memory-mapped അല്ലെങ്കിൽ isolated ആയിരിക്കാം; transfer polling, interrupts, DMA എന്നിവ ഉപയോഗിച്ച് നടത്താം.

Previously Asked Questions

Q33 (4 marks, Apr 2025) — Mention the functions of IO interface.

- *Answer:*

I/O interface CPU-യും peripheral devices-ഉം തമ്മിൽ communication സാധ്യമാക്കുന്നു. Speed, format, operation എന്നിവയിലെ വ്യത്യാസങ്ങൾക്കിടയിലും communication നടത്താൻ ഇത് സഹായിക്കുന്നു.

പ്രധാന functions:

1. Data Transfer:

Interface registers/ports ഉപയോഗിച്ച് CPU അല്ലെങ്കിൽ memory path-നും I/O devices-നും ഇടയിൽ data മാറ്റുന്നു.

2. Control Signal Management:

Peripheral operations start, stop, coordinate ചെയ്യുന്നതിനുള്ള control signals interpret ചെയ്യുകയും manage ചെയ്യുകയും ചെയ്യുന്നു.

3. Data Conversion:

CPU/memory electronics-നും electro-mechanical peripherals-നും ഇടയിലെ signal levels, data formats എന്നിവ convert ചെയ്യുന്നു.

4. Synchronisation:

Fast CPU-വും slower devices-ഉം തമ്മിലുള്ള timing match ചെയ്യുന്നു. ഇതിലൂടെ data loss അല്ലെങ്കിൽ corruption ഒഴിവാക്കുന്നു. Status checking, handshaking, buffering എന്നിവ ഇതിനായി ഉപയോഗിക്കുന്നു.

Interface ഇല്ലെങ്കിൽ ഓരോ peripheral-ന്റെയും വ്യത്യസ്ത behaviour കാരണം CPU-യുമായി direct connection practically ബുദ്ധിമുട്ടായിരിക്കും.

Q23 (2 marks, Apr 2025) — What is polling?

- *Answer:*

Polling എന്നത് ഒരു **Programmed I/O method** ആണ്. Processor ഒരു I/O device-ന്റെ status flags തുടർച്ചയായി അല്ലെങ്കിൽ ആവർത്തിച്ച് check ചെയ്യുന്നു.

Device data transfer-ന് ready ആകുമ്പോൾ CPU program control വഴി data transfer നടത്തുന്നു.

Advantage: Simple to implement.

Disadvantage: CPU waiting/checking-ൽ സമയം കളയുന്നതിനാൽ inefficient ആണ്.

Q28 (4 marks, SLM Model Set 1) — Explain the difference between synchronous and asynchronous I/O.

Point	Synchronous I/O	Asynchronous I/O
Timing	CPU/controller-നും device-നും ഇടയിൽ common clock/fixed timing ഉപയോഗിക്കുന്നു	Handshaking/status signals ഉപയോഗിക്കുന്നു; continuous shared clock ആവശ്യമില്ല
Coordination	Clocked intervals-ൽ രണ്ടും lock-step ആയി പ്രവർത്തിക്കുന്നു	Device ready/busy status നൽകുന്നു; ready ആകുമ്പോൾ CPU/interface പ്രതികരിക്കുന്നു
Speed match	Device-ന്റെയും interface-ന്റെയും speed clock-നോട് match ചെയ്യുന്നപ്പോൾ അനുയോജ്യം	Slow അല്ലെങ്കിൽ variable response time ഉള്ള devices-ന് അനുയോജ്യം
CPU involvement	Programmed timing windows ഉപയോഗിക്കാം	Status checking, interrupts അല്ലെങ്കിൽ DMA എന്നിവയുമായി സാധാരണ combine ചെയ്യാം
Risk	Clocking തെറ്റിയാൽ timing errors ഉണ്ടാകാം	ശരിയായ handshake ഇല്ലെങ്കിൽ data നഷ്ടപ്പെടുകയോ duplicate ആകുകയോ ചെയ്യാം

Summary:

Synchronous I/O transfer-ന് shared clocked timing ആശ്രയിക്കുന്നു. Asynchronous I/O units തമ്മിൽ വ്യത്യസ്ത speed ഉള്ളപ്പോഴും **handshake/status signalling** ഉപയോഗിച്ച് safe communication നടത്തുന്നു.

Interfaces-ന്റെ പ്രധാന functions-ൽ ഒന്നാണ് **synchronisation**.

Q5 (1 mark, SLM Model Set 2) — Name two types of I/O interfaces.

• *Answer:*

1. Memory-mapped I/O
2. Isolated (Port-mapped) I/O

മറ്റു valid pairs:

- Serial / Parallel
 - Programmed / Interrupt-driven
 - Synchronous / Asynchronous
-

Q37 (15 marks, SLM Model Set 2)

Analyze the role of the I/O interface in managing data transfer, control signals, data conversion, and synchronization between the CPU and peripherals, using examples from different types of I/O systems.

- *Answer:*

Introduction

Peripheral devices CPU-യിൽ നിന്ന് technology, speed, data format, operating modes എന്നിവയിൽ വ്യത്യസ്തമാണ്.

ഉദാഹരണത്തിന്, CPU electronic ആണ്, എന്നാൽ ചില peripherals electro-mechanical ആയിരിക്കാം.

I/O Interface processor bus-നും ഓരോ peripheral-നും ഇടയിൽ പ്രവർത്തിക്കുകയും ഈ വ്യത്യാസങ്ങൾക്കിടയിലും communication സാധ്യമാക്കുകയും ചെയ്യുന്നു.

1. Data Transfer

Interface registers/ports ഉപയോഗിച്ച് CPU/memory path-നും device-നും ഇടയിൽ data transfer നടത്തുന്നു.

Example:

Keyboard scan codes input data register-ൽ place ചെയ്യുന്നു. CPU ആ port-ൽ നിന്ന് data വായിക്കുന്നു.

Printer-ന് CPU output data register-ലേക്ക് എഴുതുന്ന characters ലഭിക്കുന്നു.

2. Control Signal Management

Interface control, status, data input, data output തുടങ്ങിയ commands interpret ചെയ്യുകയും device-specific control signals generate ചെയ്യുകയും ചെയ്യുന്നു.

Example:

- Disk interface seek/start-read commands നൽകാം.

- Display controller-ന് refresh അല്ലെങ്കിൽ clear ചെയ്യാൻ command നൽകാം.

3. Data Conversion

CPU/memory electronics-നും peripheral-നും ഇടയിൽ signal levels, data formats എന്നിവ convert ചെയ്യുന്നു.

Example:

- Parallel CPU data ↔ Serial line for UART
- Digital levels ↔ Printer motor drive signals

4. Synchronisation

Fast CPU-യും slower devices-ഉം തമ്മിലുള്ള timing match ചെയ്യുന്നു.

ഇതിനായി:

- Status flags
- Handshaking
- Buffering

എന്നിവ ഉപയോഗിക്കുന്നു.

Example:

CPU "ready" status bit check ചെയ്യാം (**polling**), അല്ലെങ്കിൽ അടുത്ത keyboard character വായിക്കുന്നതിന് മുമ്പ് interrupt കാത്തിരിക്കാം.

Device catch up ചെയ്യുന്നതുവരെ buffers data block hold ചെയ്യാം.

Conclusion

I/O interface ഇല്ലാതെ ഓരോ peripheral-ന്റെയും വ്യത്യസ്ത behaviour കാരണം CPU-യുമായി direct connection impractical ആയിരിക്കും.

Interfaces system-നും peripherals-നും ഇടയിൽ:

- Reliable data transfer
- Control
- Data conversion
- Synchronisation

എന്നിവ സാധ്യമാക്കുന്നു.

2.2.4–2.2.5 I/O Bus, Commands, Isolated vs Memory-Mapped I/O

Theory

I/O bus-ൽ:

- Data lines
- Address lines
- Control lines

ഉണ്ടാകും.

ഓരോ peripheral-നും ഒരു interface ഉണ്ടായിരിക്കും. ഇത് address/control decode ചെയ്യുകയും data synchronize ചെയ്യുകയും device controller-ുമായി communication നടത്തുകയും ചെയ്യുന്നു.

Processor device address address lines-ൽ place ചെയ്യുന്നു. Matching interface I/O command-ന് response നൽകുന്നു.

Four Command Types

1. **Control**
2. **Status**
3. **Data output**
4. **Data input**

Memory & I/O-യുടെ Three Bus Organisations

1. Memory-നും I/O-ക്കും separate buses.
 - ഉദാ: I/O processor / data channel
2. One common bus, separate control lines.
3. One common bus, common control lines.

Isolated I/O

- Separate I/O address space ഉണ്ടായിരിക്കും.
- Special **IN/OUT instructions** ഉപയോഗിക്കുന്നു.
- Separate I/O read/write lines ഉണ്ടായിരിക്കും.

Memory-Mapped I/O

- Interface registers memory address space-ൽ ഉൾപ്പെടുന്നു.
- സാധാരണ **load/store instructions** ഉപയോഗിച്ച് I/O നടത്താം.
- Separate I/O instructions ആവശ്യമില്ല.
- Available memory addresses കുറയുന്നു.

I/O devices connect ചെയ്യാൻ സാധാരണ ഉപയോഗിക്കുന്ന structure: Single Bus Structure.

Previously Asked Questions

Q18 (2 marks, SLM Model Set 2)

What is the purpose of a status command in an I/O interface?

- *Answer:*

Status command I/O interface-നോട് device-ന്റെ current state report ചെയ്യാൻ ആവശ്യപ്പെടുന്നു.

ഉദാഹരണങ്ങൾ:

- Ready / Busy
- Error
- Buffer Full / Empty

ഈ information ഉപയോഗിച്ച് data transfer safe ആണോ എന്ന് CPU/controller തീരുമാനിക്കുന്നു.

ഇത് data lost അല്ലെങ്കിൽ overwritten ആകുന്നത് ഒഴിവാക്കാൻ സഹായിക്കുന്നു.

Status checking programmed I/O / polling-ൽ വളരെ പ്രധാനമാണ്.

Diagram (refer SLM): Fig. 2.2.1 I/O bus to devices; Fig. 2.2.2-2.2.3 I/O Interface Unit.

Unit 3: Priority Interrupts

2.3.1–2.3.2 Priority Interrupt Concept and Types of Interrupts

Theory

Keystroke, disk ready, network packet തുടങ്ങിയ **rare, unpredictable events**-ന് fast response ആവശ്യമാണ്.

ഓരോ program-ലും polling നടത്തുന്നത് expensive ആണ്.

അതുകൊണ്ട് **interrupts** ഉപയോഗിച്ച് ആവശ്യമായ സമയത്ത് control service routine-ലേക്ക് transfer ചെയ്യുന്നു. Service പൂർത്തിയായ ശേഷം control തിരികെ വരുന്നു.

പല devices ഒരേ സമയം service request ചെയ്താൽ ഏത് request ആദ്യം service ചെയ്യണമെന്ന് തീരുമാനിക്കാൻ **Priority Interrupt System** ഉപയോഗിക്കുന്നു.

ഒരു service ഇതിനകം നടക്കുമ്പോൾ മറ്റൊരു interrupt അതിനെ interrupt ചെയ്യണമോ എന്നും ഇത് തീരുമാനിക്കുന്നു.

Fast devices, ഉദാ. magnetic disk, സാധാരണ high priority ആയിരിക്കും.

Slow devices, ഉദാ. keyboard, low priority ആയിരിക്കും.

Priority Interrupt Flow



Establishing Priority

Priority തീരുമാനിക്കാൻ:

1. **Software** — Polling
2. **Hardware** — Daisy-chain / Parallel priority

Types of Interrupts

Class	Subtype	Meaning
Hardware	Maskable	Higher-priority interrupt വന്നാൽ delay ചെയ്യാം
Hardware	Non-maskable	ഉടൻ handle ചെയ്യണം

Class	Subtype	Meaning
Software	Normal	Software instructions മൂലം ഉണ്ടാകുന്നത്
Software	Exception	Unplanned event, ഉദാ. divide by zero

For Exam

Priority interrupt system interrupt sources rank ചെയ്യുന്നു, simultaneous requests വന്നാൽ most urgent device ആദ്യം service ചെയ്യാൻ. Priority software polling വഴിയോ hardware (daisy chain or parallel encoder) വഴിയോ set ചെയ്യാം. Interrupts hardware/software, maskable/non-maskable ആയിരിക്കാം; exceptions unplanned software interrupts ആണ്.

Previously Asked Questions

Q17 (2 marks, Apr 2025) — Explain priority interrupt.

- *Answer:*

Priority interrupt എന്നത് interrupt sources-ന് priority levels നൽകുന്ന ഒരു interrupt system ആണ്.

രണ്ടോ അതിലധികമോ devices ഒരേ സമയം service request ചെയ്താൽ **highest-priority request ആദ്യം recognise ചെയ്ത് service ചെയ്യുന്നു.**

ഒരു service routine ഇതിനകം പ്രവർത്തിക്കുമ്പോൾ പുതിയ interrupt അതിനെ interrupt ചെയ്യണമോ എന്നും ഇത് തീരുമാനിക്കാം.

സാധാരണ:

Disk → High priority **Keyboard → Low priority**

Q35 (4 marks, Apr 2025) — Explain types of interrupts.

- *Answer:*

1. Hardware Interrupts

External devices generate ചെയ്യുന്ന interrupts.

ഉദാഹരണം: Key press.

Maskable:

Higher-priority interrupt വന്നാൽ delay ചെയ്യാം.

Non-maskable:

Delay ചെയ്യാൻ കഴിയില്ല; ഉടൻ process ചെയ്യണം.

2. Software Interrupts

Internal system/software activity മൂലം ഉണ്ടാകുന്ന interrupts.

Normal Software Interrupt:

Software instructions ഉപയോഗിച്ച് deliberately generate ചെയ്യുന്നത്.

ഉദാ: system calls / interrupt instructions.

Exceptions:

Execution സമയത്ത് unplanned ആയി ഉണ്ടാകുന്ന interrupts.

ഉദാ: divide by zero.

I/O organisation-ൽ **interrupt-driven I/O** എന്നും **polling** എന്നും വേർതിരിക്കാം.

Vectored interrupt service routine-ലേക്ക് നേരിട്ട് എത്തിക്കുന്ന address നൽകുന്നു.

Priority interrupt hardware simultaneous requests-ന് priority നൽകുന്നു.

Methods:

- Daisy chain
- Parallel priority encoder

Priority Interrupt Service

```
Device1 (High) ----\
Device2 -----> Priority Logic --> CPU INT
Device3 (Low) ----/
                        ↓
                      Save PC
                        ↓
                       ISR
                        ↓
                      Return
```

Diagram (refer SLM): Fig. 2.3.1 Servicing Interrupts; Fig. 2.3.2 Daisy-chain; Fig. 2.3.3 Daisy stage; Fig. 2.3.4 Parallel priority hardware; Fig. 2.3.5 ISR programs in memory.

2.3.3–2.3.6 Polling, Daisy-Chaining, Parallel Priority, Encoder

Theory

Polling — Software Priority

Interrupt വന്നാൽ ISR devices-നെ priority order-ൽ test ചെയ്യുന്നു.

Highest priority device ആദ്യം check ചെയ്യും.

Simple ആണ്, പക്ഷേ devices കൂടുതലാണെങ്കിൽ slow ആണ്.

Daisy-Chaining — Serial Hardware

Devices ഒരു chain ആയി connect ചെയ്തിരിക്കും.

Interrupt request line shared ആയിരിക്കും.

CPU ആദ്യം device-ന് **INTACK** നൽകുന്നു.

Device request ചെയ്തിട്ടില്ലെങ്കിൽ acknowledge അടുത്ത device-ലേക്ക് pass ചെയ്യുന്നു.

Signals:

- **PI**
- **PO**

Highest-priority requesting device **PO block** ചെയ്യുകയും അതിന്റെ vector address (**VAD**) data bus-ൽ place ചെയ്യുകയും ചെയ്യുന്നു.

Parallel Priority

- Interrupt register bits devices set ചെയ്യുന്നു.
- Mask register ഓരോ interrupt-ഉം enable/disable ചെയ്യുന്നു.
- AND gates priority encoder-ലേക്ക് feed ചെയ്യുന്നു.
- Priority encoder vector bits output ചെയ്യുന്നു.
- **IST** set ചെയ്യുന്നു.
- **IEN** program-ന് interrupts globally enable/disable ചെയ്യാൻ അനുവദിക്കുന്നു.

Interrupt Cycle Microoperations

IEN = 1, IST = 1 ആയിരിക്കുമ്പോൾ:

```
SP ← SP - 1
M[SP] ← PC
INTACK ← 1
PC ← VAD
IEN ← 0
↓
Fetch next instruction
(Start of ISR)
```

Software routines-ൽ ഓരോ device-നും ഒരു **ISR (Interrupt Service Routine)** ഉണ്ടായിരിക്കും.

Vector address-ൽ ഉള്ള **JMP** വഴി ISR-ലേക്ക് എത്തുന്നു.

Stack-ൽ return address സൂക്ഷിക്കുന്നു.

Prologue / epilogue mask, IST, IEN, saved registers എന്നിവ manage ചെയ്യുന്നു.

Unit 4: Direct Memory Access (DMA)

2.4.1–2.4.4 DMA Concept, Working, Modes, Advantages

Theory

Programmed I/O-യിൽ CPU ഓരോ data word-ഉം handle ചെയ്യണം.

വലിയ/high-speed transfers സമയത്ത് CPU waiting-ൽ idle ആകുന്നതിനാൽ ഇത് inefficient ആണ്.

Direct Memory Access (DMA) ഉപയോഗിച്ച് peripheral devices-ന് CPU-യുടെ ഓരോ data word intervention ഇല്ലാതെ **I/O device-നും main memory-നും ഇടയിൽ data blocks നേരിട്ട് transfer ചെയ്യാൻ** കഴിയും.

Transfer സമയത്ത് **DMA Controller (DMAC)** bus master ആയി പ്രവർത്തിക്കുന്നു.

DMA Controller Components

1. **Address Unit** — addresses + device select
2. **Control Unit**
3. **Data Count** — blocks transferred / direction

Transfer പൂർത്തിയായ ശേഷം DMA controller CPU-നെ interrupt ചെയ്യുന്നു.

Working Summary

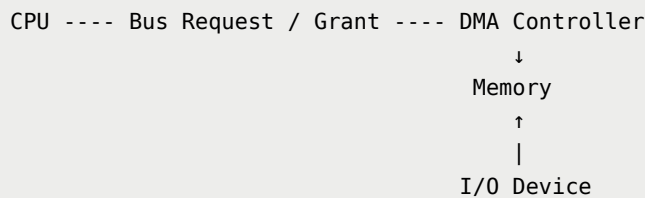
1. I/O device DMA request നൽകുന്നു അല്ലെങ്കിൽ CPU transfer setup ചെയ്യുന്നു.
2. DMAC bus request നൽകുന്നു.
3. CPU bus grant നൽകുന്നു.
4. DMAC bus master ആകുന്നു.
5. CPU memory address, count, direction എന്നിവ നൽകുന്നു.

6. DMAC memory-യും device-ഉം തമ്മിൽ data transfer ചെയ്യുന്നു.
7. Completion കഴിഞ്ഞാൽ bus release ചെയ്യുന്നു.
8. CPU-നെ interrupt ചെയ്യുന്നു.

Transfer Modes

Mode	Behaviour
Burst DMA	മുഴുവൻ block complete ആകുന്നതുവരെ bus കൈവശം വരുന്നു
Cycle Stealing DMA	ഒരു byte/word transfer ചെയ്ത ശേഷം bus തിരികെ നൽകുന്നു; വീണ്ടും repeat ചെയ്യുന്നു
Transparent DMA	CPU bus ഉപയോഗിക്കാത്ത സമയത്ത് മാത്രം bus ഉപയോഗിക്കുന്നു

DMA Diagram



DMA process:

Request → Grant → Transfer → Completion (Interrupt)

Advantages of DMA

1. **Efficient Data Transfer**
CPU byte-by-byte I/O ചെയ്യുന്നതിനെക്കാൾ faster ആണ്.
2. **CPU Offloading**
CPU-ന് മറ്റു tasks ചെയ്യാൻ കഴിയും.
3. **Higher Throughput**
Large volumes of data വേഗത്തിൽ transfer ചെയ്യാം.

ഉദാഹരണങ്ങൾ:

- Disks
- NIC
- Audio

- Graphics

Disadvantages

- Bus contention
- Extra hardware complexity
- CPU-യും DMA-യും same memory unsafe ആയി access ചെയ്താൽ data coherency issues

Applications

- Disk / SSD
- Network cards
- Sound cards
- Graphics transfers

For Exam

DMA I/O devices-ന് DMA controller temporarily bus master ആയി ഉപയോഗിച്ച് memory-യിലേക്ക്/മുതൽ data directly transfer ചെയ്യാൻ അനുവദിക്കുന്നു. CPU transfer setup മാത്രം ചെയ്യുന്നു; completion കഴിഞ്ഞാൽ interrupt ലഭിക്കുന്നു. Modes: burst, cycle stealing, transparent. Advantages: speed, less CPU overhead, bulk transfers-ന് high throughput.

Previously Asked Questions

Q9 (1 mark, Apr 2025)

DMA stands for _____.

- *Answer:* Direct Memory Access.**

Q5 (1 mark, SLM Model Set 1)

What does DMA stand for in I/O systems?

- *Answer:* Direct Memory Access.**

Q37 (15 marks, Apr 2025) — Explain the concept of Direct Memory Access (DMA). Discuss the advantages of using DMA in a computer system.

- *Answer:*

Concept of DMA

Direct Memory Access എന്നത് peripheral devices-ന് CPU ഓരോ byte-നും continuous involvement ഇല്ലാതെ **main memory-യിൽ നിന്ന് data എടുക്കാനും main memory-യിലേക്ക് data നൽകാനും** അനുവദിക്കുന്ന technique ആണ്.

DMA controller addresses, word count, bus control എന്നിവ manage ചെയ്യുന്നു.

CPU പല I/O paths-നേക്കാൾ fast ആയതിനാൽ programmed I/O ഉപയോഗിക്കുമ്പോൾ CPU busy ആകുകയോ waiting-ൽ idle ആകുകയോ ചെയ്യും. അതിനാൽ DMA system efficiency മെച്ചപ്പെടുത്തുന്നു.

Need

Disk, network, multimedia തുടങ്ങിയ high-speed devices-ൽ large blocks CPU program control വഴി transfer ചെയ്യുന്നത് slow-വും wasteful-ഉം ആണ്.

DMA block transfers-ന് **"bus master"** ആയി പ്രവർത്തിക്കുന്നു.

Working

1. Device അല്ലെങ്കിൽ CPU DMA request ചെയ്യുന്നു.
2. DMA controller system bus request ചെയ്യുന്നു.
3. CPU bus grant നൽകുന്നു.
4. CPU DMAC-നെ starting memory address, number of words/blocks, transfer direction എന്നിവ ഉപയോഗിച്ച് initialise ചെയ്യുന്നു.
5. DMAC bus master ആയി memory-യും I/O-യും തമ്മിൽ data transfer ചെയ്യുന്നു.
6. Mode അനുസരിച്ച് CPU മറ്റ് instructions continue ചെയ്യാം.
7. Count zero ആയാൽ DMAC bus release ചെയ്യുകയും CPU-നെ interrupt ചെയ്യുകയും ചെയ്യുന്നു.

DMA Transfer Modes

Burst Mode:

മുഴുവൻ block complete ആകുന്നതുവരെ bus hold ചെയ്യുന്നു.

Cycle Stealing:

ഓരോ byte/word കഴിഞ്ഞും bus return ചെയ്യുന്നു.

Transparent Mode:

CPU bus ആവശ്യമില്ലാത്ത സമയത്ത് മാത്രം transfer ചെയ്യുന്നു.

Block Path

I/O Device ↔ DMA Controller ↔ System Bus ↔ Main Memory

ഓരോ word-നും CPU data path-ൽ ഉണ്ടാകേണ്ടതില്ല.

Advantages

1. CPU-driven programmed I/O-നെക്കാൾ efficient/high-speed transfer.
2. CPU offloading — DMA data move ചെയ്യുമ്പോൾ processor computation/multitasking ചെയ്യാം.
3. Bulk data-യ്ക്ക് higher throughput.
4. ഓരോ byte-നും interrupt നൽകേണ്ടതില്ല; block complete ആയപ്പോൾ ഒരു interrupt മതിയാകും.

DMA-യ്ക്ക് extra hardware, bus contention ഒഴിവാക്കാനുള്ള synchronisation എന്നിവ ആവശ്യമാണ്.

എന്നിരുന്നാലും large transfers-ൽ DMA-യുടെ performance benefits കൂടുതലാണ്.

Q28 (4 marks, SLM Model Set 2) — How does Direct Memory Access (DMA) improve system performance in data transfer operations?

- *Answer:*

DMA devices-ന് I/O-യും main memory-യും തമ്മിൽ data blocks നേരിട്ട് transfer ചെയ്യാൻ അനുവദിക്കുന്നു.

DMA controller **bus master** ആയി പ്രവർത്തിക്കുന്നതിനാൽ CPU ഓരോ byte-ഉം സ്വയം move ചെയ്യേണ്ടതില്ല.

Performance Gains

1. Programmed I/O-നെക്കാൾ faster bulk transfer.
2. CPU offloading — DMA data transfer ചെയ്യുമ്പോൾ CPU useful work ചെയ്യാം.
3. Disk, NIC, audio, graphics എന്നിവയിൽ higher I/O throughput.
4. Fewer interrupts — ഓരോ byte-നും പകരം ഓരോ block-നും ഒരു completion interrupt.

Modes:

- Burst
- Cycle stealing
- Transparent

ഇവ bus occupancy-യും CPU progress-ഉം തമ്മിൽ trade-off നൽകുന്നു.

Large transfers-ൽ overall system throughput വർദ്ധിക്കുന്നു.

Diagram (refer SLM): Fig. 2.4.1 DMA Controller Architecture; Fig. 2.4.2 Block diagram of DMA Controller; Fig. 2.4.3 Data transfer by DMA.

Block 3: Parallel Computer Structures

Unit 1: Introduction to Parallel Processing

3.1.1 Parallel Processing Concepts / Serial vs Parallel

Theory

ഒരു computer data input ആയി സ്വീകരിക്കുന്നു, instructions അനുസരിച്ച് process ചെയ്യുന്നു, output produce ചെയ്യുന്നു, അത് memory-ൽ store ചെയ്യുന്നു.

Instructions:

- Sequential ആയി
- Parallel ആയി

execute ചെയ്യാം.

Sequential / Serial Processing

Sequential approach-ൽ CPU ഒരു സമയത്ത് ~~ഒരു~~ instruction മാത്രം fetch ചെയ്ത് execute ചെയ്യുന്നു.

ഉദാഹരണം:

```
Instruction 1
  ↓
Instruction 2
  ↓
Instruction 3
  ↓
Instruction 4
```

ഒരു instruction finish ചെയ്തതിന് ശേഷമാണ് അടുത്ത instruction തുടങ്ങുന്നത്.

അതുകൊണ്ട് overall performance slow ആകാം.

Parallel Processing

Parallel processing processing capability വർദ്ധിപ്പിക്കുകയും data-processing throughput കൂട്ടുകയും ചെയ്യുന്നു.

Several processing units ഒരുമിച്ച് പ്രവർത്തിച്ച് multiple tasks/instructions concurrent ആയി execute ചെയ്യുന്നു.

Important Points

- **Serial = One-by-one execution**
- **Parallel = Concurrent execution**

Parallel processing-ന്റെ ലക്ഷ്യം:

- Computation speed വർദ്ധിപ്പിക്കുക
- Throughput വർദ്ധിപ്പിക്കുക

Parallel Architecture Configurations

1. Pipeline computers
2. Array processors
3. Multiprocessor systems

Parallel programming, serial programming-നെക്കാൾ complex ആണ്.

Massive / big-data computational work-ന് parallel processing അനുയോജ്യമാണ്.

For Exam

Serial computing single flow of control-ൽ instructions one after another execute ചെയ്യുന്നതിനാൽ കൂടുതൽ time എടുക്കുന്നു. Parallel computing multiple processing units ഉപയോഗിച്ച് multiple tasks/instructions simultaneously execute ചെയ്യുന്നു; performance and throughput improve ചെയ്യുന്നു. Parallel architectures include pipeline computers, array processors, multiprocessor systems.

Previously Asked Questions

Q7 (1 mark, SLM Model Set 2)

In which type of processing are several instructions executed simultaneously?

- *Answer: Parallel Processing.***

Pipelining-ലും multiple instructions different stages-ൽ overlap ചെയ്യും.

Diagram (refer SLM): Fig 3.1.1 Serial Vs Parallel Processing; Fig 3.1.5 Serial processing (one billing counter); Fig 3.1.6 Parallel processing (multiple counters)

3.1.2–3.1.3 Data Processing Cycle

Theory

Data Processing Cycle raw data-യെ usable information ആക്കി മാറ്റുന്നു.

Stages:

1. **Data Collection** — raw data ശേഖരിക്കുന്നു.
2. **Data Preparation** — data desired form-ലേക്ക് manipulate ചെയ്യുന്നു.
3. **Data Entry** — verified data input devices വഴി system-ലേക്ക് നൽകുന്നു.
4. **Data Processing** — methodologies ഉപയോഗിച്ച് data process ചെയ്യുന്നു.
5. **Data Interpretation** — meaningful information derive ചെയ്യുന്നു.
6. **Data Storage** — instructions/information future use-നായി store ചെയ്യുന്നു.

Important Points

Six stages:

Collection → Preparation → Entry → Processing → Interpretation → Storage

```

Raw Data
  ↓
Collection
  ↓
Preparation
  ↓
Entry
  ↓
Processing
  ↓
Interpretation
  ↓
Storage
  
```

Raw data → processed information → memory-ൽ storage.

For Exam

Data processing cycle: raw data collect ചെയ്യുക, prepare ചെയ്യുക, system-ലേക്ക് enter ചെയ്യുക, process ചെയ്യുക, results interpret ചെയ്യുക, later use-നായി store ചെയ്യുക.

Diagram (refer SLM): Fig 3.1.2 Data Processing Cycle; Fig 3.1.3 Data transfer I/O to CPU; Fig 3.1.4 Steps in Data Processing

3.1.7 Pipeline Computers

Theory

Pipeline computers CPU hardware ഇങ്ങനെ arrange ചെയ്യുന്നു, overall performance വർദ്ധിപ്പിക്കാൻ.

Multiple instructions pipeline fashion-ൽ execute ചെയ്യുന്നു.

ഒരു stage-ന്റെ output അടുത്ത stage-ന്റെ input ആകുന്നു.

ഓരോ segment-ലും സാധാരണ:

- Input register
- Combinational circuit

ഉണ്ടാകും.

Common clock ഓരോ clock-ലും data-യെ ഒരു step മുന്നോട്ട് കൊണ്ടുപോകുന്നു.

Example

$(A_i B_i + C_i)$ calculate ചെയ്യുന്നത് മൂന്ന് segments ആയി:

1. Input / multiply operands
2. Multiply + input C_i
3. Add

Pipeline fill ചെയ്ത ശേഷം ഓരോ subsequent clock-ലും പുതിയ result ലഭിക്കും.

Formula

k-segment pipeline-ൽ n tasks process ചെയ്യുമ്പോൾ:

Total time = $(k + n - 1)$ clock cycles

അല്ലെങ്കിൽ:

Total time = $(k + (n-1)) \times t_p$

Important Points

- Stages overlap ചെയ്യുന്നു.

- Multiple instructions ഒരേസമയം pipeline-ൽ ഉണ്ടായിരിക്കും.
- Space-time diagram pipeline utilisation കാണിക്കുന്നു.
- First result $\rightarrow k$ cycles കഴിഞ്ഞ്.
- അതിന് ശേഷം ideal ആയി ഓരോ cycle-ലും ഒരു result.

For Exam

Pipelining instruction/computation stages ആയി divide ചെയ്യുന്നു, അതിനാൽ several instructions overlap ചെയ്യുന്നു. Fill time കഴിഞ്ഞാൽ throughput one result per clock cycle-നോട് approach ചെയ്യും. k stages-ൽ n tasks-നുള്ള time ഏകദേശം $k + (n-1)$ cycles ആണ്.

Diagram (refer SLM): Fig 3.1.7 Hardware arrangements; Fig 3.1.8 $A_i B_i + C_i$; Fig 3.1.9 Four-segment pipeline; Fig 3.1.10 Space-Time Diagram

3.1.8–3.1.9 Array Processors and Multiprocessing

Theory

Array Processor

ഒരു instruction ഉപയോഗിച്ച് പല processing elements-ലും ഒരേസമയം execution നടത്താൻ കഴിയുന്ന multiprocessor-style organisation ആണ് **Array Processor**.

ഇത്:

- Arrays
- Vectors
- Floating-point numbers

എന്നിവയിലെ arithmetic operations-ന് അനുയോജ്യമാണ്.

Multiprocessing System

ഒരു multiprocessing system-ൽ ഒന്നിലധികം CPUs main memory-യും peripherals-ഉം share ചെയ്യുന്നു.

Types

1. Shared-memory multiprocessor

Processors shared physical address space ഉപയോഗിക്കുന്നു.

2. Private-memory multiprocessor

ഓരോ processor-നും private, non-shared address space ഉണ്ടായിരിക്കും.

Interconnection Structures

- Time-shared common bus
- Multiport memory
- Crossbar switch
- Multistage switching network
- Hypercube

Important Points

Array Processor $\approx$ SIMD-style vector/array arithmetic

Multiprocessor = Multiple CPUs, often shared memory

Interconnects:

- Bus
- Multiport
- Crossbar
- Multistage
- Hypercube

For Exam

Array processors one instruction many data elements-ൽ apply ചെയ്യുന്നു (arrays/vectors). Multiprocessors multiple CPUs use ചെയ്ത് memory/peripherals share ചെയ്ത് large volumes quickly process ചെയ്യുന്നു. Interconnection choices (bus, multiport, crossbar, multistage, hypercube) bandwidth and contention affect ചെയ്യുന്നു.

Previously Asked Questions

Q24 (2 marks, Apr 2025) — Difference between parallel and serial computing

- *Answer:*

Serial computing:

Instructions/tasks ഒന്നിന് ശേഷം മറ്റൊന്ന് sequence-ൽ execute ചെയ്യുന്നു.

CPU instruction 1 complete ചെയ്ത ശേഷമാണ് instruction 2 തുടങ്ങുന്നത്.

അതുകൊണ്ട് large workloads-ൽ performance slow ആണ്.

Parallel computing:

Multiple processing units ഒരമിച്ച് പ്രവർത്തിച്ച് multiple instructions/data items ഒരേ സമയം process ചെയ്യുന്നു.

Work processors-കിടയിൽ divide ചെയ്യുന്നു.

ഇത്:

- Throughput വർദ്ധിപ്പിക്കുന്നു
- Wall-clock time കുറയ്ക്കുന്നു

Serial solutions implement ചെയ്യാൻ simpler ആണ്.

Parallel solutions program ചെയ്യാൻ കൂടുതൽ difficult ആണ്, പക്ഷേ big-data/high-performance applications-ന് കൂടുതൽ അനുയോജ്യമാണ്.

Parallel organisations:

- Pipeline computers
- Array processors
- Multiprocessor systems

Diagram (refer SLM): Fig 3.1.11 Vector Addition; Fig 3.1.12 Multiprocessing system; Fig 3.1.13-3.1.18 Bus / Multiport / Crossbar / Omega / Hypercube

Unit 2: Architectural Classification (Flynn's and Related)

3.2.1–3.2.2 Classification Schemes Overview

Theory

Parallel computers വലിയ problems-നെ ചെറിയ ഭാഗങ്ങളായി divide ചെയ്ത് multiple instructions/calculations ഒരേ സമയത്ത് execute ചെയ്യുന്നു.

പ്രധാനപ്പെട്ട മൂന്ന് classical classification schemes:

Scientist	Year	Basis
Flynn	1966	Instruction & Data streams-ന്റെ multiplicity
Feng	1972	Degree of parallelism — bit/word level
Handler	1977	PCU / ALU / BLC levels-ലെ pipelining & parallelism

ഇതുകൂടാതെ:

- **Coupling** → Loose coupling / Tight coupling
- **Memory access mode** → UMA / NUMA

Important Points

- **Flynn classification** ആണ് exam-ൽ ഏറ്റവും widely used taxonomy.
- **Instruction stream** = instructions-ന്റെ sequence.
- **Data stream** = memory-യും processing unit-ഉം തമ്മിലുള്ള data flow/traffic.

For Exam

Architectural classification schemes computers-നെ instructions/data എങ്ങനെ flow ചെയ്യുന്നു, bits/words എങ്ങനെ parallel process ചെയ്യുന്നു, processors എങ്ങനെ coupled ആണ്, memory എങ്ങനെ access ചെയ്യുന്നു എന്നീ അടിസ്ഥാനങ്ങളിൽ group ചെയ്യുന്നു. Flynn's taxonomy (SISD/SIMD/MISD/MIMD) standard exam answer ആണ്.

3.2.3 Flynn's Classification

Theory

M. J. Flynn (1966) instruction stream-ഉം data stream-ഉം **single അല്ലെങ്കിൽ multiple** ആണോ എന്നതിന്റെ അടിസ്ഥാനത്തിൽ computer architectures-നെ classify ചെയ്തു.

നാല് types:

1. **SISD** — Single Instruction, Single Data
2. **SIMD** — Single Instruction, Multiple Data
3. **MISD** — Multiple Instruction, Single Data
4. **MIMD** — Multiple Instruction, Multiple Data

1. SISD — Single Instruction, Single Data

ഒരു conventional **uniprocessor von Neumann machine** ആണ് SISD.

- One instruction stream
- One data stream
- One CPU
- ഒരേ സമയം ഒരു instruction ഒരു data item-ൽ execute ചെയ്യുന്നു.

Example

Traditional personal computers / early sequential computers.

2. SIMD — Single Instruction, Multiple Data

ഒരു **Control Unit** ഒരേ instruction പല identical **Processing Elements (PEs)**-ലേക്ക് broadcast ചെയ്യുന്നു.

ഓരോ PE-യും വ്യത്യസ്ത data-യിൽ അതേ instruction execute ചെയ്യുന്നു.

- One instruction
- Many data
- Synchronous
- Deterministic

Examples

- Array processors
 - Vector/array machines
 - ILLIAC IV style
 - GPU-style SIMD workloads
-

3. MISD — Multiple Instruction, Single Data

Multiple processors **different instruction streams** ഒരേ data stream-ൽ execute ചെയ്യുന്നു.

Data പല processors-ലൂടെ flow ചെയ്യുന്നു.

Important

- Commercial machines-ൽ വളരെ rare ആണ്.
 - Flynn ഈ category predict ചെയ്തിരുന്നു.
 - Pure MISD systems വളരെ കുറച്ച് മാത്രമാണ് നിർമ്മിച്ചിട്ടുള്ളത്.
 - Specialised pipelined / fault-tolerant designs-മായി ചിലപ്പോൾ associated ആണ്.
-

4. MIMD — Multiple Instruction, Multiple Data

Multiple processors multiple memory modules-മായി **interconnection network** വഴി connected ആയിരിക്കും.

ഓരോ Processing Element-നും:

- Different instruction execute ചെയ്യാം.
- Different data process ചെയ്യാം.

Execution:

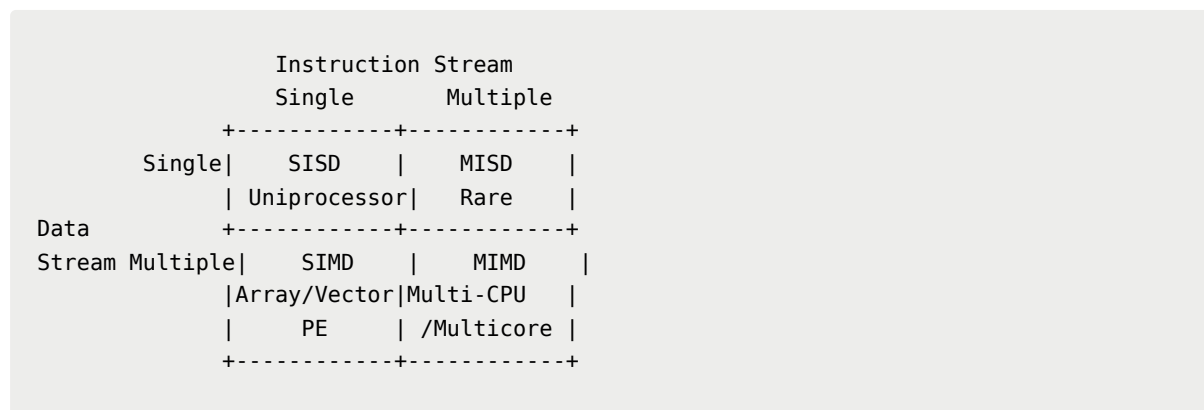
- Synchronous അല്ലെങ്കിൽ asynchronous
- Deterministic അല്ലെങ്കിൽ non-deterministic

ആയിരിക്കാം.

Examples

- Modern multiprocessors
- Multicore SMPs
- Clusters
- Loosely coupled MIMD systems

Diagram — Flynn's Classification



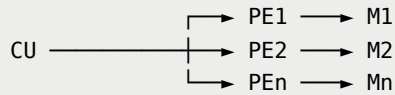
Structure Sketch

SISD

CU → PE → Memory

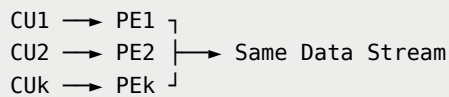
One instruction + one data stream.

SIMD

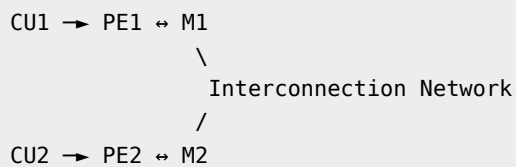


Same instruction, different data.

MISD



MIMD



Multiple instructions + multiple data.

Important Points

SISD

Traditional serial computer

SIMD

One instruction → Many data

Array / Vector processing.

MISD

Many instructions → One data

Least common.

MIMD

Many instructions → Many data

General multiprocessor systems.

Note: Flynn classification-ൽ എല്ലാ classes-ഉം parallel അല്ല. **SISD is serial.**

For Exam

Flynn computers-നെ instruction-stream and data-stream multiplicity അനുസരിച്ച് SISD, SIMD, MISD, MIMD ആയി classify ചെയ്യുന്നു. SISD classic single-CPU von Neumann machine ആണ്. SIMD one control unit and many PEs different data-ൽ ഉപയോഗിക്കുന്നു (array processors). MISD one data stream multiple instruction streams വഴി feed ചെയ്യുന്നു (rare). MIMD multiple independent instruction/data streams run ചെയ്യുന്നു (multiprocessors/multicores). 2x2 taxonomy draw ചെയ്ത് ഓരോന്നിനും example നൽകണം.

Previously Asked Questions

Q38 (15 marks, Apr 2025)

Explain Flynn's computer system classification with examples.

- *Answer:*

Introduction: 1966-ൽ Michael J. Flynn computer architectures-നെ **instruction streams-ഉം data streams-ഉം** എത്രയാണെന്ന് അടിസ്ഥാനമാക്കി classify ചെയ്തു.

Instruction stream എന്നത് processing unit execute ചെയ്യുന്ന instructions-ന്റെ sequence ആണ്. **Data stream** എന്നത് memory-ക്കും processor-നും ഇടയിൽ operands/data flow ചെയ്യുന്നതാണ്.

Instruction stream single അല്ലെങ്കിൽ multiple ആയിരിക്കാം. Data stream single അല്ലെങ്കിൽ multiple ആയിരിക്കാം. അതിനാൽ നാല് classes ലഭിക്കുന്നു:

1. **SISD**
2. **SIMD**
3. **MISD**
4. **MIMD**

1. SISD — Single Instruction Single Data

SISD ഒരു **uniprocessor von Neumann machine** ആണ്. ഒരു CPU ഒരേ സമയം ഒരു instruction ഒരു data item-ൽ execute ചെയ്യുന്നു. Instructions and data primary memory-ൽ ഉണ്ടായിരിക്കും.

Example: Conventional PCs, early sequential computers.

Advantages: Implement ചെയ്യാൻ simple ആണ്; ഓരോ clock cycle-ലും one instruction stream and one data stream active ആണ്.

2. SIMD — Single Instruction Multiple Data

SIMD-ൽ front-end **Control Unit** ഒരേ instruction പല identical, synchronised **Processing Elements (PEs)**-ലേക്ക് issue ചെയ്യുന്നു. ഓരോ PE-യും different data-യിൽ അതേ instruction apply ചെയ്യുന്നു. Memory often modular ആയിരിക്കും, അതിനാൽ എല്ലാ PEs-നും data ലഭിക്കും.

Execution synchronous and deterministic ആണ്.

Example: Array processors, vector-oriented machines such as ILLIAC IV, modern data-parallel / GPU-style SIMD workloads.

Advantages: Same instruction many data elements-ൽ ഒരേ സമയം apply ചെയ്യാം; arrays/numerical work-ന് suitable ആണ്.

3. MISD — Multiple Instruction Single Data

MISD-ൽ several processors **different instruction streams** ഒരേ **data stream**-ൽ execute ചെയ്യുന്നു. Data different processing stages-യുടെ pipeline പോലെ flow ചെയ്യുന്നു.

Example: Pure MISD commercial machines വളരെ കുറവാണ്; mostly theoretical category ആണ്. ചില specialised/fault-tolerant/cascaded processing designs-ഉം മായി associated ആയി കാണാം.

Advantages: One data path-ൽ multiple independent instruction streams ഉപയോഗിക്കാം.

4. MIMD — Multiple Instruction Multiple Data

MIMD-ൽ multiple processors and memory modules **interconnection network** വഴി communicate ചെയ്യുന്നു. ഓരോ processor-നും different program different data-യിൽ run ചെയ്യാം. Operation synchronous അല്ലെങ്കിൽ asynchronous ആയിരിക്കാം.

Example: Shared-memory multiprocessors, multicore CPUs, loosely coupled clusters.

Advantages: Flexible general-purpose parallelism; multiple instruction streams and multiple data streams support ചെയ്യുന്നു.

Conclusion: Flynn classification exam-ൽ standard architectural taxonomy ആണ്. **SISD serial** ആണ്. Practical parallel systems-ൽ **SIMD** and **MIMD** കൂടുതലായി

കാണുന്നു. **MISD** rare ആണ്. Exam answer-ൽ instruction/data streams define ചെയ്ത് നാല് classes diagrams/examples സഹിതം എഴുതണം.

Q7 (1 mark, SLM Model Set 1)

What does SIMD stand for?

- *Answer:*

Single Instruction Multiple Data.

Q6 (1 mark, SLM Model Set 2)

What are uniprocessing computing devices called?

- *Answer:*

SISD machines / Uniprocessors

അത്:

Single Instruction Stream + Single Data Stream

ഉള്ള conventional single-CPU computers ആണ്.

Q36 (15 marks, SLM Model Set 2)

Evaluate Flynn's classification of parallel processing with necessary diagrams.

- *Answer:* — Short Exam Format

Introduction:

M. J. Flynn (1966) instruction streams-ഉം data streams-ഉം single/multiple ആണോ എന്നതിന്റെ അടിസ്ഥാനത്തിൽ computers-നെ നാല് classes ആയി classify ചെയ്തു:

1. SISD
2. SIMD
3. MISD
4. MIMD

Classification

	Single Instruction	Multiple Instruction
Single Data	SISD	MISD

	Single Instruction	Multiple Instruction
Multiple Data	SIMD	MIMD

Conclusion

- **SISD** → Serial
- **SIMD** → Array/Vector processing
- **MIMD** → Practical multiprocessor parallelism
- **MISD** → Rare

Exam-ൽ **classification table + architecture type-ന്റെ diagram + example** എഴുതുന്നത് നല്ലതാണ്.

Diagram (refer SLM): Fig 3.2.2 Instruction and data stream; Fig 3.2.3 Taxonomy of Flynn's Classification; Fig 3.2.4 SISD; Fig 3.2.5 SIMD; Fig 3.2.6 MISD; Fig 3.2.7 MIMD

3.2.4 Feng's Classification

Theory

Feng (1972) computers-നെ **word-level and bit-level parallelism**-ന്റെ അടിസ്ഥാനത്തിൽ classify ചെയ്യുന്നു.

നാല് types:

1. **WSBS — Word Serial Bit Serial**
2. **WPBS — Word Parallel Bit Serial**
3. **WSBP — Word Serial Bit Parallel**
4. **WPBP — Word Parallel Bit Parallel**

1. WSBS — Word Serial Bit Serial

- **Word Serial**
- **Bit Serial**
- ഒരു സമയത്ത് ഒരു bit വീതം process ചെയ്യുന്നു.

അതിനാൽ parallelism വളരെ കുറവാണ്.

2. WPBS — Word Parallel Bit Serial

- **Word Parallel**
- **Bit Serial**
- ഇത് **bit-slice** processing ആയി കാണാം.
- ഒരു സമയത്ത് **m-bit slice** process ചെയ്യുന്നു.

3. WSBP — Word Serial Bit Parallel

- **Word Serial**
- **Bit Parallel**
- ഒരു സമയത്ത് ഒരു **n-bit word** process ചെയ്യുന്നു.
- Most conventional computers ഈ രീതിയുമായി ബന്ധപ്പെട്ടതാണ്.

4. WPBP — Word Parallel Bit Parallel

- **Word Parallel**
- **Bit Parallel**
- Fully parallel processing ആണ്.
- ഒരേ സമയം **(n × m) bits** വരെ process ചെയ്യാം.

Important Points

Degree of parallelism എന്നത് ഒരു unit time-ൽ process ചെയ്യാൻ കഴിയുന്ന maximum binary digits-ന്റെ അളവാണ്.

Feng classification-ൽ:

	BIT	
	Serial	Parallel
WORD		
Serial	WSBS	WSBP
Parallel	WPBS	WPBP

Easy Memory Trick

WSBS → Word Serial + Bit Serial
WPBS → Word Parallel + Bit Serial

WSBP → Word Serial + Bit Parallel
WPBP → Word Parallel + Bit Parallel

WPBP = Fully Parallel

For Exam

Feng's scheme words and bits serial/parallel ആയി process ചെയ്യുന്നതിന്റെ അടിസ്ഥാനത്തിലാണ്: WSBS, WPBS, WSBP, WPBP. WPBP fully parallel ആണ്.

Diagram (refer SLM): Fig 3.2.8 Processor classified according to Feng's Classification

3.2.5 Handler's Classification

Theory

Handler (1977) computer architecture-യിലെ **pipelining and parallelism**-നെ അടിസ്ഥാനമാക്കി classification നടത്തുന്നു.

ഇത് മൂന്ന് levels പരിഗണിക്കുന്നു:

1. **PCU — Program Control Unit**
2. **ALU — Arithmetic Logic Unit**
3. **BLC — Bit-Level Circuit**

Architecture-ൽ parallelism ഏത് level-ൽ എത്രത്തോളം ഉണ്ടെന്ന് Handler classification കാണിക്കുന്നു.

Important Points

Handler classification-ൽ പ്രധാനമായി നോക്കുന്നത്:

- Program Control Unit level parallelism
- ALU level parallelism
- Bit-level circuit parallelism
- Pipelining

അതായത് processor-ന്റെ different functional levels-ൽ parallel processing എങ്ങനെ achieve ചെയ്യുന്നു എന്നതാണ് പ്രധാന focus.

Other Classification Schemes

Architectural classification-ൽ Flynn, Feng, Handler എന്നിവയ്ക്ക് പുറമേ **coupling** and **memory access** അടിസ്ഥാനമാക്കിയുള്ള classifications-ഉം ഉണ്ട്.

1. Coupling

Processors തമ്മിലുള്ള connection/interaction അനുസരിച്ച്:

Loosely Coupled

Processors relatively independent ആയിരിക്കും.

ഓരോ processor-നും സ്വന്തം local/private memory ഉണ്ടായിരിക്കാം.

Tightly Coupled

Multiple processors shared resources, പ്രത്യേകിച്ച് **shared memory**, ഉപയോഗിക്കുന്നു.

Processors തമ്മിലുള്ള communication കൂടുതലായിരിക്കും.

2. Memory Access Mode

Memory access അനുസരിച്ച് പ്രധാനമായും:

UMA — Uniform Memory Access

എല്ലാ processors-ക്കും shared memory-യിലെ memory locations access ചെയ്യാൻ ഏകദേശം **same access time** ആണ്.

NUMA — Non-Uniform Memory Access

Memory access time processor-ന്റെ location / memory location അനുസരിച്ച് വ്യത്യാസപ്പെടാം.

അടുത്ത memory faster ആയിരിക്കാം; remote memory slower ആയിരിക്കാം.

Important Points

- **Loose coupling** = distributed systems / local memories.
- **Tight coupling** = shared common memory.
- **UMA** = uniform latency.
- **NUMA** = non-uniform latency.

For Exam

Handler PCU/ALU/BLC pipelining model ചെയ്യുന്നു. Loosely coupled systems private memory + network ഉപയോഗിക്കുന്നു; tightly coupled systems memory share ചെയ്യുന്നു. UMA equal memory latency നൽകുന്നു; NUMA local access remote access-നെക്കാൾ faster ആക്കുന്നു.

Diagram (refer SLM): Fig 3.2.9 Loosely coupled; Fig 3.2.10 Tightly coupled

Previously Asked Questions

Q29 (4 marks, SLM Model Set 2)

Compare UMA and NUMA multiprocessors.

- *Answer:*

Point	UMA (Uniform Memory Access)	NUMA (Non-Uniform Memory Access)
Memory view	എല്ലാ processors-ക്കും shared memory equal access time -ൽ ലഭിക്കുന്നു	Access time location അനുസരിച്ച് മാറുന്നു; local memory faster, remote memory slower
Typical system	Shared bus/crossbar to common memory ഉള്ള Symmetric Multiprocessors (SMP)	Distributed memory modules ഉള്ള scalable multiprocessors
Latency	Uniform / predictable	Non-uniform; remote accesses കൂടുതൽ cost/time എടുക്കും
Scalability	പല CPUs വരെ scale ചെയ്യാൻ ബുദ്ധിമുട്ട്; bus/memory contention ഉണ്ടാകാം	Large systems-ൽ better scalability
Programming	Simpler shared-memory model	More complex; locality-aware placement performance മെച്ചപ്പെടുത്തും
Coupling	Usually tightly coupled shared memory	Distributed shared memory / ccNUMA variants

Summary: UMA all CPUs-ക്കും equal memory latency നൽകുന്നു, അതിനാൽ simpler ആണ് പക്ഷേ limited scale ആണ്. NUMA local memory access faster ആക്കുന്നു, scalability മെച്ചപ്പെടുത്തുന്നു, പക്ഷേ programming complexity വർദ്ധിപ്പിക്കുന്നു.

Unit 3: Pipelining

3.3.1–3.3.2 Introduction to Pipelining

Theory

Pipelining എന്നത് factory assembly line അല്ലെങ്കിൽ water pipe പോലെ പ്രവർത്തിക്കുന്ന ഒരു technique ആണ്.

ഒരു ജോലി ഒരു stage-ൽ നടക്കുമ്പോൾ, മറ്റൊരു ജോലി അടുത്ത/മറ്റൊരു stage-ൽ simultaneously നടക്കുന്നു.

അതായത്, **multiple instructions execution overlap ചെയ്യുന്നു.**

Typical Five-Stage Pipeline

Fetch → Decode → Compute → Memory → Write

ഓരോ instruction-നും പല cycles ആവശ്യമാണ്.

പക്ഷേ pipeline fill ആയതിന് ശേഷം, ideal condition-ൽ **ഒരു cycle-ൽ ഒരു instruction complete** ചെയ്യാൻ കഴിയും.

Data Hazard

ഒരു instruction-ന് ആവശ്യമായ operand data available അല്ലാത്തപ്പോൾ **data hazard** ഉണ്ടാകുന്നു.

Advantages of Pipelining

- Instruction throughput വർദ്ധിക്കുന്നു.
- കൂടുതൽ stages → കൂടുതൽ instructions overlap ചെയ്യാം.
- ALU design കൂടുതൽ efficient ആകാം.
- Higher clock frequency ലഭിക്കാം.
- Overall CPU performance മെച്ചപ്പെടുന്നു.

Disadvantages

- Design കൂടുതൽ complex ആണ്.
- Instruction latency വർദ്ധിക്കാം.
- Throughput predict ചെയ്യുന്നത് difficult ആകാം.
- Long pipelines-ൽ branch hazards കൂടുതൽ പ്രശ്നമാകും.

Important Points

Pipelining ≠ Multiple independent CPUs

Pipelining-ൽ ഒരു processor-ന്റെ instruction/computation stages **overlap** ചെയ്യുകയാണ്.

Pipeline construction-നെ ബാധിക്കുന്ന factors:

1. Level of processing
2. Pipeline configuration
3. Type of instruction/data

For Exam

Pipelining instruction execution stages overlap ചെയ്യുന്നു, അതിനാൽ several instructions ഒരു സമയം progress-ൽ ഉണ്ടായിരിക്കും; throughput increase ചെയ്യും. Fill time കഴിഞ്ഞാൽ completion rate one instruction per cycle-നോട് approach ചെയ്യും, hazards പ്രത്യേകിച്ച് branches and data dependencies ഇതിനെ affect ചെയ്യും.

Previously Asked Questions

Q10 — 1 mark, Apr 2025

What do you mean by pipelining?

- *Answer:*

Multiple instructions-ന്റെ execution **overlap** ചെയ്യുന്ന implementation technique ആണ് pipelining.

Instruction processing stages ആയി divide ചെയ്യുന്നു:

Fetch → Decode → Execute → Memory → Write-back

ഒരു instruction ഒരു stage-ൽ process ചെയ്യുമ്പോൾ മറ്റൊരു instruction മറ്റൊരു stage-ൽ process ചെയ്യും.

ഇത് **instruction throughput** വർദ്ധിപ്പിക്കുന്നു.

Q8 — 1 mark, SLM Model Set 1

“Cycle time of the processor is reduced” is one of the disadvantages of Pipelining. True/False

- *Answer:*

False

Processor cycle time കുറയ്ക്കുകയും throughput improve ചെയ്യുകയും ചെയ്യുന്നത് **advantage** ആണ്.

Disadvantages:

- Complex design
- Data/control hazards
- Pipeline stalls/bubbles
- Branch handling difficulty

Q29 — 4 marks, SLM Model Set 1

What is pipelining? Define processor cycle in pipelining.

- *Answer:*

Pipelining successive instructions-ന്റെ execution stages **overlap** ചെയ്യുന്ന technique ആണ്.

Stages:

Fetch → Decode → Execute → Memory → Write-back

Pipeline fill ആയ ശേഷം ideal ആയി **one instruction per clock cycle** complete ചെയ്യാം.

Processor Cycle

Pipeline cycle / clock period എന്നത് ഒരു task pipeline-ലെ ഒരു **stage** മുറുന്ന് പോകാൻ എടുക്കുന്ന സമയം ആണ്.

ഇത് സാധാരണയായി:

Slowest stage + latch overhead

ആണ് determine ചെയ്യുന്നത്.

ഒരു cycle-ൽ ഓരോ occupied stage-ഉം അതിന്റെ micro-operation നടത്തുകയും result അടുത്ത stage-ലേക്ക് move ചെയ്യുകയും ചെയ്യുന്നു.

Formula

k stages, n tasks:

$$\text{Total time} \approx [k + (n - 1)] \times t_p$$

Pipelining instruction rate മെച്ചപ്പെടുത്തുന്നു, പക്ഷേ hazards eliminate ചെയ്യുന്നില്ല.

Q30 — 4 marks, SLM Model Set 1

State the need for Instruction Level Parallelism (ILP).

- *Answer:*

Instruction Level Parallelism (ILP) എന്നത് ഒരേ instruction stream-ൽ നിന്നുള്ള multiple instructions **concurrently execute/overlap** ചെയ്യുന്നതാണ്.

ILP ആവശ്യമായത്:

1. **Higher performance** — sequential completion കാത്തിരിക്കാതെ performance വർദ്ധിപ്പിക്കാൻ.
2. **Better CPU utilisation** — ALU, memory ports തുടങ്ങിയ functional units idle ആകാതെ ഉപയോഗിക്കാൻ.
3. **Increased throughput** — unit time-ൽ കൂടുതൽ instructions complete ചെയ്യാൻ.
4. **Independent instructions** — independent instructions ഒരമിച്ച് schedule ചെയ്യാൻ.
5. **Modern high-speed CPUs** — clock speed മാത്രം വർദ്ധിപ്പിച്ച് performance കൂട്ടുന്നതിനുള്ള limitations മറികടക്കാൻ.

ILP ഇല്ലെങ്കിൽ instructions serial ആയി execute ചെയ്യേണ്ടിവരുന്നതിനാൽ CPU-യുടെ potential concurrency waste ആകും.

Q19 — 2 marks, SLM Model Set 2

What is meant by the pipeline bubble?

- *Answer:*

ഒരു instruction മുന്നോട്ട് പോകാൻ കഴിയാത്തപ്പോൾ pipeline-ൽ insert ചെയ്യുന്ന **empty stage cycle** ആണ് pipeline bubble.

ഇത് സംഭവിക്കാൻ കാരണങ്ങൾ:

- Data hazard
- Control/branch hazard
- Structural conflict

ആ stage ആ clock cycle-ൽ useful work ഒന്നും ചെയ്യില്ല.

അതുകൊണ്ട് effective throughput കുറയും.

Diagram (refer SLM): Fig 3.3.1 Water pipe analogy

3.3.3 Classification by Level of Processing

Theory

1. Instruction Execution Pipeline

Instruction processing ഒരു **assembly line** പോലെ നടത്തുന്നു.

Simplest Form

രണ്ട് stages:

1. **Fetch**
2. **Execute**

ഒരു instruction execute ചെയ്യുമ്പോൾ അടുത്ത instruction fetch ചെയ്യാം.

Six-Stage Instruction Pipeline

Common six-stage model:

1. **IF — Instruction Fetch**
2. **ID — Instruction Decode**
3. **AG — Address Generator**
4. **DF — Data Fetch**
5. **EX — Execution**
6. **WB — Write-back**

Alternative Six-Phase Breakdown

- **FI**
- **DI**
- **CO**
- **FO**
- **EI**
- **WO**

Six-stage pipeline നിരവധി instructions-ന്റെ execution time വളരെ കുറയ്ക്കാം.

പക്ഷേ conditional branch ഉണ്ടാകുമ്പോൾ useless instructions flush ചെയ്യേണ്ടി വരുകയും pipeline stall ആകുകയും ചെയ്യാം.

2. Arithmetic Operation Pipeline

Arithmetic operation pipeline ഉപയോഗിക്കുന്നത്:

- Floating-point operations
- Fixed-point multiplication
- മറ്റു arithmetic operations

എന്നിവയാണ്.

Floating-Point Addition/Subtraction — Four Segments

1. Compare exponents
2. Align mantissas
3. Add/Subtract mantissas
4. Produce normalised result

Pipeline Diagram

IF → ID → AG → DF → EX → WB

Example:

Clock →	1	2	3	4	5	6	7
I1	IF	ID	AG	DF	EX	WB	
I2		IF	ID	AG	DF	EX	WB
I3			IF	ID	AG	DF	EX

Important Points

Instruction Pipeline:

Successive instructions-ന്റെ:

Fetch / Decode / Execute

stages overlap ചെയ്യുന്നു.

Arithmetic Pipeline:

Numeric computation-ന്റെ sub-operations overlap ചെയ്യുന്നു.

Branches and interrupts handle ചെയ്യാൻ special pipeline logic ആവശ്യമാണ്.

For Exam

Instruction pipelines successive instructions IF-ID-AG-DF-EX-WB stages across overlap ചെയ്യുന്നു. Arithmetic pipelines floating-point substeps overlap ചെയ്യുന്നു: compare exponents, align, add/subtract, result. Prefetch സഹായിക്കുന്നു, പക്ഷേ unequal stage times and branches ideal 2x speedup limit ചെയ്യുന്നു.

Diagram (refer SLM): Fig 3.3.2 Two-stage pipeline; Fig 3.3.3 Timing diagram; Fig 3.3.4 Conditional branch effect; Fig 3.3.5 Six-stage CPU instruction pipeline; Fig 3.3.6 FP add/sub pipeline

3.3.4–3.3.5 Configuration and Instruction/Data Types

Theory

By Configuration

1. Unifunction Pipeline

ഒരു fixed/dedicated function മാത്രം തുടർച്ചയായി perform ചെയ്യുന്നു.

Unifunction = Specialised

2. Multifunction Pipeline

Different times-ൽ different functions perform ചെയ്യാൻ കഴിയും.

Multifunction = Flexible

By Instruction/Data Type

1. Scalar Pipeline

Individual operands-ൽ repeated scalar instructions process ചെയ്യുന്നു.

2. Vector Pipeline

Vector operands / arrays-ൽ vector instructions process ചെയ്യുന്നു.

Vector pipelines high-performance numerical computing-ൽ പ്രധാനമാണ്.

Important Points

Unifunction → Specialised

Multifunction → Flexible

Scalar → Individual operands

Vector → Arrays / Vector operands

For Exam

Pipelines unifunction അല്ലെങ്കിൽ multifunction ആയിരിക്കാം, കൂടാതെ scalar അല്ലെങ്കിൽ vector operands process ചെയ്യും. Vector pipelines high-performance numerical computing-ൽ key ആണ്.

Previously Asked Questions

Q20 — 2 marks, SLM Model Set 1

Mention the various types of pipelining.

- *Answer:*

Pipelining-നെ പല രീതിയിൽ classify ചെയ്യാം:

1. By Level of Processing

- Instruction Execution Pipeline
- Arithmetic Operation Pipeline

2. By Configuration

- Unifunction Pipeline
- Multifunction Pipeline

3. By Instruction/Data Type

- Scalar Pipeline
- Vector Pipeline

Short answer:

Instruction + Arithmetic + Unifunction + Multifunction + Scalar + Vector

Unit 4: Vector Processing and Array Processors

3.4.1 Vector Processing

Theory

Vector എന്നത് one-dimensional ordered collection of data items ആണ്.

Example:

$V = [V1, V2, \dots, Vn]$

Vector processor എന്നത് vectors/arrays-ൽ operate ചെയ്യുന്ന instructions ഉള്ള CPU ആണ്. സാധാരണ floating-point data-യിലാണ് ഇത് കൂടുതലായി ഉപയോഗിക്കുന്നത്.

ഒരു instruction ഉപയോഗിച്ച് multiple data items-ൽ operation നടത്തുന്നതിനെ **SIMD / vector (array) instructions** എന്നും പറയുന്നു. Vector data **vector registers**-ൽ store ചെയ്യാം. ഒരേ operation different data elements-ൽ repeat ചെയ്യപ്പെടുന്നു.

Scalar processor: ഒരു സമയത്ത് ഒരു item മാത്രം process ചെയ്യുന്നു. Integers/floats sequential ആയി process ചെയ്യും. Large arrays-ൽ ഇത് slower ആണ്.

Vector processor: പല data points aggregate ചെയ്ത് ഒരേ operation efficiently apply ചെയ്യുന്നു. പക്ഷേ memory data supply ചെയ്യാൻ കഴിയുന്നില്ലെങ്കിൽ system-ന്റെ other parts stress ആകാം.

Applications: long-range weather forecasting, petroleum exploration, medical diagnosis, aerodynamics simulations, AI/expert systems, image processing.

Matrix multiply heavy vector workload-ന്റെ classic example ആണ്. $n \times n$ matrices multiply ചെയ്യാൻ നിരവധി inner products / multiply-add operations വേണം. Pipeline vector processors multiplier and adder pipelines ഉപയോഗിച്ച് inner products compute ചെയ്യുന്നു.

Memory interleaving: Memory പല modules ആയി split ചെയ്യുന്നു. ഓരോ module-ക്കും സ്വന്തം AR/DR ഉണ്ടാകും. അതിനാൽ instruction + operand അല്ലെങ്കിൽ multiple vector operands ഒരേ സമയം access ചെയ്യാൻ കഴിയും. ഇത് effective memory cycle time modules-ന്റെ number അനുസരിച്ച് കുറയ്ക്കാൻ സഹായിക്കുന്നു.

Important Points

- Vector = 1-D array.
- Vector CPU = vectors-ൽ operate ചെയ്യുന്ന instructions ഉള്ള CPU.
- Setup time = vector functional unit-ലേക്ക് route ചെയ്യാൻ വേണ്ട time.

- Flushing time = vector instruction start ചെയ്തതിൽ നിന്ന് first result pipeline-ൽ നിന്ന് പുറത്തുവരുന്നതുവരെ വേണ്ട time.
- Interleaved memory pipelines/vector units-ന് data feed ചെയ്യുന്നു.

For Exam

Vector processing scalar loops പോലെ ഓരോ element process ചെയ്യുന്നതിനപകരം entire arrays/vectors-ൽ same arithmetic operation execute ചെയ്യുന്നു. Vector instructions and often pipelined floating-point units ഉപയോഗിക്കുന്നു. Scientific computing-ൽ ഇത് vital ആണ്: weather, simulation, imaging. Performance setup, flushing, memory bandwidth/interleaving, compiler/algorithm choices എന്നിവയിൽ depend ചെയ്യുന്നു.

Diagram (refer SLM): Fig 3.4.1 Vector instruction format; Fig 3.4.2 Inner-product pipeline; Fig 3.4.3 Multiple-module memory

Previously Asked Questions

Q5 (1 mark, Apr 2025)

What is Vector processing?

- *Answer:* Vector processing ഒരു parallel data-processing method ആണ്. ഇതിൽ CPU / vector processor **one-dimensional arrays (vectors)** of data-ൽ operate ചെയ്യുന്ന instructions execute ചെയ്യുന്നു. സാധാരണ ഒരേ operation പല elements-ൽ apply ചെയ്യുന്നു; ഒരു scalar value മാത്രം process ചെയ്യുന്നതല്ല.

3.4.4 Performance Measures of Vector Processing

Theory

Vector processing-ൽ പ്രധാനപ്പെട്ട രണ്ട് time overheads ഉണ്ട്:

1. **Setup time** — vector functional unit-ലേക്ക് route ചെയ്യാൻ വേണ്ട time.
2. **Flushing time** — vector instruction-ന്റെ initial processing level മുതൽ **first result** pipeline-ൽ നിന്ന് emerge ചെയ്യുന്നതുവരെ വേണ്ട duration.

Performance improve ചെയ്യാൻ SLM പറയുന്ന നാല് measures/practices:

1. **Improve the vector instruction** — memory access കുറയ്ക്കുക; resource utilisation maximise ചെയ്യുക.
2. **Integrate scalar instructions** — same type scalar instructions batch ചെയ്യുക; pipeline repeatedly reconfigure ചെയ്യേണ്ട overhead കുറയ്ക്കാൻ.
3. **Algorithm** — vector pipelines-ന് well map ചെയ്യുന്ന algorithms തിരഞ്ഞെടുക്കുക.

4. **Vectorising compiler** — high-level language-ൽ നിന്ന് parallelism regenerate ചെയ്യുക. Development stages: Parallel Algorithm (A) → High-level Language (L) → Efficient Object code (O) → Target Machine code (M).

Supercomputers-ുമായി ബന്ധപ്പെട്ട metric: **FLOPS** — floating-point operations per second. Megaflops / Gigaflops ഉപയോഗിക്കുന്നു. Cray-1 പോലുള്ള supercomputers vector instructions and pipelined FP units combine ചെയ്യുന്നു.

Important Points

- Setup + flushing steady pipeline throughput തുടങ്ങുന്നതിന് മുൻപുള്ള key overheads ആണ്.
- Four exam points: better vector instructions, integrate scalars, good algorithm, vectorising compiler.
- A-L-O-M = stages of parallelism development.

For Exam

Vector processing performance setup and flushing times കൊണ്ട് limited ആകുന്നു. Measures: vector instructions improve ചെയ്യുക (less memory traffic, better utilisation), similar scalar work batch ചെയ്യുക, vector-friendly algorithms select ചെയ്യുക, vectorising compiler use ചെയ്യുക (A → L → O → M). Pipeline results produce ചെയ്ത ശേഷം sustained rate often megaflops-ൽ quoted ചെയ്യും.

Previously Asked Questions

Q32 (4 marks, Apr 2025)

Explain the Performance Measures of Vector Processing.

- *Answer:* Vector processing performance pipeline overheads-ിലും software/hardware vector units busy ആയി keep ചെയ്യുന്ന രീതിയിലും ആശ്രയിക്കുന്നു.

Overheads:

1. **Setup time** — vector operands functional unit-ലേക്ക് route ചെയ്യാൻ വേണ്ട time.
2. **Flushing time** — vector instruction start ചെയ്തതിൽ നിന്ന് first result pipeline-ൽ നിന്ന് പുറത്തുവരുന്നതുവരെ വേണ്ട time. Pipeline fill ആകുന്നതുവരെ useful results delayed ആയിരിക്കും.

Performance improve ചെയ്യാനുള്ള measures/practices:

1. **Improving the vector instruction** — memory accesses കുറയ്ക്കുകയും functional units/registers maximum ഉപയോഗിക്കുകയും ചെയ്യുക.

2. **Integrating scalar instructions** — same type scalar instructions group ചെയ്യുക, അതിനാൽ pipeline repeatedly reconfigure ചെയ്യേണ്ട overhead കുറയും.
3. **Algorithm selection** — long vectors and regular operations expose ചെയ്യുന്ന, pipelined vector hardware-ന് suitable ആയ algorithms തിരഞ്ഞെടുക്കുക.
4. **Vectorising compiler** — high-level code-ൽ നിന്ന് parallelism recover ചെയ്ത് efficient vector code generate ചെയ്യുന്ന compiler. Parallelism development: Parallel Algorithm (A) → High-level Language (L) → Efficient Object code (O) → Machine code (M).

ഇവ pipeline idle time കുറയ്ക്കുകയും sustained floating-point throughput വർദ്ധിപ്പിക്കുകയും ചെയ്യുന്നു.

3.4.5-3.4.8 Supercomputers and Array Processors

Theory

Supercomputer high-speed scientific work-നായി vector instructions and pipelined floating-point arithmetic commercially combine ചെയ്യുന്ന computer ആണ്. Weather forecasting, seismic analysis, space research തുടങ്ങിയ applications-ൽ ഉപയോഗിക്കുന്നു. Dense packaging and cooling critical ആണ്.

Example: Cray-1 (1976) — vector processing, 12 functional units, pipeline വഴി ഏകദേശം 80 megaflops peak.

Array processor: large arrays-ൽ computations നടത്തുന്ന processor ആണ്. SLM usage-ൽ ഇത് multiprocessor/vector processor എന്നും കാണാം.

Array processors രണ്ട് types:

1. **Attached array processor** — host general-purpose computer-ന്റെ peripheral ആയി പ്രവർത്തിക്കുന്നു. I/O interface + local memory ഉണ്ടായിരിക്കും. Pipelined floating-point units ഉപയോഗിച്ച് numerical/vector work accelerate ചെയ്യുന്നു.
2. **SIMD array processor** — one control unit + many PEs with local memories. One instruction stream, multiple data streams. Example: ILLIAC IV.

Why use array processors? Higher instruction processing speed ലഭിക്കുന്നു; host-ൽ നിന്ന് often asynchronous ആയി പ്രവർത്തിക്കുന്നതിനാൽ better capacity ലഭിക്കുന്നു; local memory capacity add ചെയ്യുന്നു.

Limitation: Data items തമ്മിൽ dependency ഉണ്ടെങ്കിൽ fully parallel ആയി execute ചെയ്യാൻ കഴിയില്ല. A finish ചെയ്താൽ മാത്രമേ B start ചെയ്യാൻ കഴിയൂ എങ്കിൽ dependency array parallelism limit ചെയ്യും.

Important Points

- Attached AP vs SIMD AP പ്രധാന distinction ആണ്.
- Data dependency ആണ് array processing-ന്റെ major limitation.

For Exam

Array processors large array/matrix computations attached accelerators ആയി അല്ലെങ്കിൽ one control unit കീഴിലുള്ള SIMD machines ആയി speed up ചെയ്യുന്നു. Data dependencies sequential order force ചെയ്താൽ അവ fully parallelise ചെയ്യാൻ കഴിയില്ല.

Diagram (refer SLM): Fig 3.4.4 Attached array processor; Fig 3.4.5 SIMD array processor organisation

Previously Asked Questions

Q6 (1 mark, SLM Model Set 1)

Who is considered the father of vector processing and supercomputing?

- *Answer: Seymour Cray.*

Block 3 — Quick Revision

Block 3-ൽ പ്രധാനമായി ഓർക്കേണ്ടത്:

Unit 1 — Basic Computer Organisation

- Computer organisation
- CPU
- Memory
- I/O
- Instruction execution

Unit 2 — Computer Architecture Classification

- Flynn classification
- SISD
- SIMD
- MISD

- MIMD
- UMA
- NUMA
- Tightly coupled / Loosely coupled

Unit 3 — Pipelining

- Pipelining
- Pipeline stages
- Instruction pipeline
- Arithmetic pipeline
- Pipeline bubble
- ILP
- Unifunction / Multifunction
- Scalar / Vector pipeline

Unit 4 — Vector Processing

- Vector processing
- Scalar vs Vector
- Setup time
- Flushing time
- Memory interleaving
- Performance measures
- Supercomputer
- Array processor
- Attached array processor
- SIMD array processor
- Data dependency
- **Seymour Cray**

ഇതൊരു PDF പ്രകാരം Block 3 അവസാനിക്കുന്നു. അടുത്തത് Block 4 – Basic Concepts of Operating Systems ആണ്.